

Università degli Studi di Padova

DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”

MASTER’S DEGREE IN COMPUTER SCIENCE



**An Experimental Methodology for
Performance Evaluation and Capacity
Planning of Cloud-Based Healthcare
Interoperability Gateways**

Master Thesis

Relatore

Prof. Vardanega Tullio

Laureando

Angeloni Alberto

Matricola 2131060

ANNO ACCADEMICO 2025-2026

“Colui il quale ha inseguito e sconfitto i demoni Sem, che ora vagano per il mondo, domandandosi: «ma nû, chi sëm?»”

— Il grande Pdor, figlio di Kmer, della tribù di Ishtar, della terra desolata dei Kfnir, uno degli ultimi sette saggi: Pfulur, Galér, Astaparigna, Sùsar, Param, Fusus e Tarìm.

Ringraziamenti

Desidero esprimere la mia gratitudine al professor Vardanega Tullio, mio relatore, per l'aiuto e il sostegno che mi ha dato durante la stesura dell'elaborato.

Vorrei anche ringraziare, con affetto, i miei genitori per il loro sostegno, il grande aiuto e la loro presenza in ogni momento durante gli anni di studio.

Desidero poi ringraziare i miei amici per i bellissimi anni trascorsi insieme e le mille avventure vissute.

Padova, September 2026

Angeloni Alberto

Abstract

The increasing digitalization of community-based healthcare services requires information architectures capable of ensuring interoperability, information continuity, and integration among heterogeneous systems. This thesis is framed within the Territorial Information System for the Healthcare Organizations of the Veneto Region, a project aimed at reducing local application fragmentation and supporting community care processes in line with Ministerial Decree 77/2022 and the Italian National Recovery and Resilience Plan. The work, carried out during an internship at Al maviva in the digital healthcare domain, addresses healthcare interoperability from a dual perspective: a technical-functional analysis of regional integration flows and the design of an original Python framework for the end-to-end simulation of RVE application traffic. The framework supports synthetic FHIR patient generation, multi-step workload composition, stochastic traffic scheduling, asynchronous execution in mock and live modes, structured logging, and performance-oriented metric computation. The proposed methodology adopts a two-level grey-box approach, distinguishing framework-observable measurements from middleware-level evidence, and provides a methodological basis for translating healthcare interoperability processes into executable and measurable experimental models. Community-based healthcare depends on the ability of heterogeneous systems to exchange consistent information across organizational, technological, and clinical boundaries. In the Veneto regional context, this problem emerges from the coexistence of clinical-care applications, authentication services, regional patient registry services, Electronic Health Record components, integration middleware, and domain-specific interoperability specifications. The central issue addressed by this thesis is therefore not limited to message exchange: it concerns the modelling, execution, and observation of multi-step healthcare interoperability flows whose correctness and performance depend on several coordinated systems. This creates the need for a methodology that connec-

ts functional analysis with experimental evaluation. The thesis pursues two complementary objectives. The first is analytical: to study the healthcare and integration context of community care processes, with particular attention to protected discharge and protected admission, regional RVE transactions, IHE profiles, HL7 FHIR R4, and security mechanisms based on SAML and JWT. The second is technical: to design and implement a framework capable of generating synthetic patients, composing executable RVE workloads, scheduling traffic over time, executing flows in controlled conditions, and collecting data suitable for later performance analysis. Together, these objectives support a structured bridge between process analysis and measurable interoperability behavior. The work combines documentary analysis, flow modelling, software implementation, and experimental-methodology design. On the domain side, it considers regional healthcare interoperability specifications, HL7 FHIR R4 resources, IHE profiles, SOAP and REST communication patterns, XML and JSON representations, SAML assertions, and JWT-based access mechanisms. On the technical side, the implemented framework is based on Python and includes modules for FHIR patient synthesis, RVE flow composition, stochastic timeline generation, asynchronous HTTP execution through `asyncio` and `aiohttp`, mock and live execution modes, JSONL logging, log normalization, and metric computation. These components are used to represent healthcare flows as executable workloads. The main contribution of the thesis lies in the integration of two perspectives that are often treated separately. The analytical contribution consists in reconstructing relevant healthcare interoperability scenarios and identifying how clinical-care processes can be mapped onto regional transactions, standards, and integration layers. The technical contribution consists in the design and implementation of a simulation and measurement framework that turns these flows into executable workloads. This makes it possible to observe step-level and flow-level behavior, distinguish framework-side and middleware-side evidence, and prepare an experimental basis for evaluating latency, throughput, error rate, concurrency, and workload composition.

0.1 Results

(DATA MISSING)

0.2 Conclusions

(DATA MISSING)

Indice

Elenco delle figure

Elenco delle tabelle

Elenco dei codici sorgenti

5.1	Example of code	82
-----	---------------------------	----

Acronimi e abbreviazioni

UML Unified Modeling Language. [78](#)

Glossario

Nome del termine Descrizione. [78](#)

Introduction

1.1 Reference Scenario

Healthcare information systems increasingly support care processes that cross organizational, technological, and clinical boundaries. This is especially relevant in community-based healthcare, where continuity of care depends on the coordinated exchange of administrative, clinical, and operational information among heterogeneous applications. In the Veneto Region, this need is framed by the reorganization of territorial care promoted by Ministerial Decree 77/2022 and by the development of the Territorial Information System for regional Healthcare Organizations. In this context, interoperability is not only a technical problem of connecting services. It also requires interpretation of healthcare processes, the alignment of regional specifications, the use of national and international standards, and the ability to observe how integration flows behave under realistic workload conditions. This thesis therefore follows a dual track. The first track is analytical and functional: it studies the healthcare domain, the actors involved, and the integration flows relevant to community care. The second track is technical and experimental: it designs a framework for modelling, executing, and measuring end-to-end RVE application traffic in a controlled environment. This chapter introduces the healthcare, industrial, and research context of the work. It presents the internship setting, the regional SI.Ter initiative, the research problem, the thesis objectives, the main contributions, and the structure of the dissertation, without anticipating the detailed design, implementation, or experimental results developed in later chapters.

1.1.1 Thesis Context

The thesis originates from an industrial internship carried out at Al maviva in the digital healthcare domain. The internship provided access to a project

context in which software integration, regional healthcare specifications, and organizational requirements are closely connected. The work is therefore positioned at the intersection between applied system integration and academic research on interoperability and performance evaluation. The central observation behind the thesis is that healthcare interoperability cannot be understood only by reading protocol specifications or only by testing isolated endpoints. In regional healthcare systems, each application flow is embedded in a broader organizational process and often involves authentication, patient identification, document exchange, middleware mediation, and application-specific business logic. For this reason, the thesis combines technical-functional analysis with the design of an executable simulation framework. This section introduces the industrial and methodological background of the work. It first frames the internship experience, then clarifies the role of the system integrator, and finally identifies the available evidence about the involvement in the RTI team and in the technical-functional analysis activities.

1.1.1.1 The internship in the Digital Healthcare Domain

The thesis was developed from an internship at Al MAVIVA, in a professional context related to digital healthcare and system integration. The work was connected to the broader SI.Ter initiative, the Territorial Information System for the Healthcare Organizations of the Veneto Region. The project documentation describes SI.Ter as a regional initiative involving clinical-care modules, interoperability services, authentication mechanisms, patient registry services, document repositories, and integration with external systems. This setting provided a concrete environment in which to observe how healthcare processes are translated into software integration requirements. The internship did not consist only in the study of generic healthcare standards. Its relevance for the thesis lies in the exposure to a real interoperability ecosystem, where technical specifications have to be interpreted in relation to organizational processes and application constraints. The documents consulted during the work include the

SI.Ter technical tender documentation, the RTI technical report, regional specifications for Anagrafe Zero, context-call mechanisms, security infrastructure, and interoperability profiles. These sources show that the regional healthcare environment combines multiple interaction styles, including REST APIs, SOAP-based services, FHIR resources, XDS.b document exchange, SAML assertions, and JWT-based access tokens. Within this context, the thesis focuses on the parts of the internship that are academically relevant: the analysis of integration flows and the design of a framework capable of reproducing their execution logic in a controlled setting. This choice avoids treating the internship as a descriptive account of company activity and instead uses it as the starting point for a research and engineering problem: how to represent healthcare interoperability flows as executable, observable, and measurable workloads. Some details of the personal internship activity still require direct confirmation before being used as final thesis evidence. (DATA MISSING: precise internship dates, formal role title, team name, internal deliverables produced, and activities validated by company tutor.) Until these details are confirmed, this section should remain focused on the documented project context and on the thesis work that can be verified through available documentation and the current codebase.

1.1.1.2 System Integrators in Healthcare Interoperability Projects

In healthcare interoperability projects, the system integrator operates between organizational requirements, existing applications, external suppliers, regional specifications, and technical infrastructure. This role is not equivalent to the ownership of a single product module, nor to the operational management of the whole production environment. Its central task is to make heterogeneous systems cooperate according to shared interfaces, security constraints, data models, and process requirements. The SI.Ter documentation makes this role particularly visible. The proposed solution is described as a multi-layer architecture in which application modules, interoperability components, workflow automation, persistence, monitoring, and external systems cooperate through

APIs, web services, message queues, and transformation logic. The integration layer is responsible for coordinating flows between internal modules and external systems, supporting standards and formats such as REST, HL7, FHIR, JSON, XML, and XDS.b. This makes interoperability a mediation activity: it requires routing, transformation, validation, error handling, traceability, and alignment with the semantics of the healthcare process. From the thesis point of view, the system integrator's role is important because it exposes the gap between formal specifications and executable behavior. A transaction specification can define the expected structure of a request or response, but a real flow often depends on a sequence of steps: authentication, retrieval or generation of patient context, invocation of an application service, possible document publication, and logging or audit. The integrator must therefore reason about dependencies between systems and about the operational consequences of failures, timeouts, incomplete data, or inconsistent identifiers. This perspective motivates the technical contribution of the thesis. If interoperability is mediated by multiple layers, then performance evaluation cannot be reduced to a single endpoint measurement. It requires a model of the flow, its dependencies, its workload composition, and the observability points available at different architectural levels.

1.1.1.3 Involvement in the RTI

The SI.Ter project documentation identifies the RTI as the organizational context responsible for the proposed solution and describes the coexistence of several application modules, suppliers, and integration components. The available evidence supports the statement that the thesis work was developed in relation to this project environment and that the analysis concerned technical-functional aspects of healthcare interoperability. However, the exact operational involvement of the student within the RTI team is not fully documented in the sources currently available. For this reason, this section must remain controlled. It can state that the thesis builds on the study of project documentation, regional

specifications, and implementation evidence from the developed framework. It can also state that the analysis of flows contributed to the identification of use cases such as protected discharge and protected admission, where patient identification, care-setting transitions, access to clinical information, and integration with regional services become relevant. What cannot yet be stated without further evidence is the precise set of meetings attended, internal artefacts produced, responsibilities assigned, or interactions with specific RTI members. (DATA MISSING: documented description of the student's activities in the RTI team, validated by tutor or company documentation.) The question to resolve before finalizing this section is: which concrete technical-functional analysis activities were carried out during the internship, and which of them can be described in the thesis without exposing confidential project details? Until this evidence is available, the academically safe formulation is that the thesis uses the RTI project context as a case study and derives from it a methodological need: translating healthcare interoperability flows into formal workload models that can be executed and measured.

1.1.2 Regional Healthcare Context

The regional healthcare context is relevant because the thesis does not study interoperability as an abstract software problem. It studies it in relation to territorial care, regional governance, and the need to coordinate services distributed across Healthcare Organizations. Ministerial Decree 77/2022 frames the development of territorial care in the Italian National Health Service and defines organizational and technological standards that increase the need for digital continuity across care settings. In the Veneto Region, this context is connected to the SI.Ter initiative, whose purpose is to support territorial healthcare services through a regional information system. The coexistence of local applications, regional platforms, authentication services, patient registry systems, Electronic Health Record components, and document repositories makes interoperability a structural requirement rather than an optional feature.

This section introduces the regional healthcare setting, the national regulation on territorial care, the link with Mission 6 of the National Recovery and Resilience Plan, and the problem of information fragmentation in community-based services.

1.1.2.1 The Healthcare System of Regione Veneto

For the purposes of this thesis, the Veneto healthcare system is considered through the lens of the actors and systems that participate in interoperability flows. The relevant level is not a general institutional history of the regional health service, but the interaction between Azienda Zero, the regional Healthcare Organizations, local care services, and shared digital infrastructures. The SI.Ter documentation positions the project within the ecosystem of the Healthcare Organizations of the Veneto Region. The system is required to interact with existing regional and local services, including authentication infrastructure, Anagrafe Zero for patient registry functions, Electronic Health Record services, XDS repositories, prescription services, CUP systems, ADT systems, digital signature providers, payment systems, and other administrative or logistical components. This gives the regional healthcare system a distributed structure: some capabilities are centralized or regional, while others remain connected to the individual Healthcare Organizations and their existing applications. From an interoperability perspective, this organization has direct consequences. A single care process may require data from multiple sources and may involve several responsibility boundaries. For example, patient identification can involve regional registry services; clinical documentation can involve the Electronic Health Record and XDS repositories; access can depend on authentication and authorization services; and territorial workflows can involve local application modules. The relevant technical problem is therefore the orchestration of heterogeneous systems, each with its own interface, data model, and operational constraints. This is the context in which SI.Ter becomes important for the thesis. It is not treated merely as an application suite, but

as a case study of regional healthcare integration. Its role is to make visible the connection between healthcare organization, software architecture, and the need for controlled experimental methods to evaluate integration flows.

1.1.2.2 PNRR

Ministerial Decree 77/2022 defines models and standards for the development of territorial care within the Italian National Health Service. The decree distinguishes between a descriptive model of territorial care and a set of prescriptive standards, requiring regional planning to be aligned with the standards connected to Mission 6, Component 1 of the National Recovery and Resilience Plan. For this thesis, the decree is relevant because it makes territorial care a coordinated system of services rather than a collection of isolated local activities. The decree identifies several structures and functions that are significant for interoperability. The District is described as a coordination level for territorial health and social-health services. The Casa della Comunita is a physical access point for citizens and multidisciplinary care. The Centrale Operativa Territoriale, or COT, coordinates patient transitions across care settings and requires information systems capable of tracking and monitoring those transitions. The 116117 operational center provides access to non-urgent medical care and must be connected with territorial coordination mechanisms. Home care, community hospitals, telemedicine, and other territorial services also require digital continuity. The technological implications are explicit in several parts of the regulation. Territorial coordination depends on systems able to collect, manage, and monitor health data, access the Electronic Health Record, interconnect with company applications, and support telemedicine or shared information flows. Therefore, the regulation does not only define organizational structures; it also creates a demand for interoperable information systems. In the thesis, DM 77/2022 provides the healthcare rationale for the technical work. If territorial care requires coordination among services, then integration flows must be studied not only for correctness but also for behavior under load, observa-

bility, and robustness. This link justifies the transition from domain analysis to workload generation and performance-oriented experimentation. The available documentation supports a direct link between Ministerial Decree 77/2022 and Mission 6 of the National Recovery and Resilience Plan. The decree is described as a milestone for the reform of territorial care, and the extracted documentation reports investments connected to home care, Centrali Operative Territoriali, and telemedicine. This connection is important because it shows that territorial healthcare digitalization is part of a broader national transformation program, not an isolated regional initiative. For the thesis, the relevant aspect is not the financial detail of the investment plan, but the operational consequence: territorial care requires shared, timely, and reliable information. The COT must coordinate transitions between settings; home care requires information exchange with territorial platforms and the Electronic Health Record; telemedicine requires digital infrastructures that support remote interaction and data collection. These requirements increase the dependency on interoperable systems. The SI.Ter initiative can be read within this transformation context. It aims to support territorial healthcare services by providing application modules and integration capabilities that can cooperate with regional and local systems. The need for interoperability is therefore generated by both organizational reform and technical heterogeneity. The decree itself is sufficient to justify the connection between territorial-care reform and Mission 6, Component 1 of the National Recovery and Resilience Plan. Exact investment-line figures are not needed for the introductory argument and should be introduced only if the final bibliography includes the corresponding institutional source. [DATA MISSING: BibLaTeX entry and exact institutional source for PNRR investment-line figures or milestone tables, if those details are retained later in the thesis.]

1.1.2.3 Information Fragmentation in Territorial Healthcare

Information fragmentation is one of the central motivations for SI.Ter and for this thesis. It can be understood at three levels. Application fragmentation occurs when different services or Healthcare Organizations use different software systems. Information fragmentation occurs when patient, clinical, administrative, or workflow data are distributed across registries, local applications, document repositories, and regional platforms. Organizational fragmentation occurs when care processes require coordination among actors that belong to different services or settings. In territorial healthcare, these forms of fragmentation are particularly relevant because care continuity depends on transitions. A patient may move from hospital discharge to home care, from a territorial access point to a COT evaluation, or from an application module to an Electronic Health Record consultation. Each transition requires the correct identification of the patient, the availability of relevant clinical information, and the consistent execution of authorization and audit mechanisms. The SI.Ter documentation explicitly places the solution in continuity with systems already in use and describes the need to integrate with multiple existing components. This confirms that the problem is not simply to replace legacy systems, but to make heterogeneous systems cooperate. The thesis takes this fragmentation as the starting point for its research problem. If the same care process depends on multiple systems and protocols, then evaluation must consider the behavior of the flow as a whole, not only the behavior of an isolated service call.

1.1.3 SI.Ter: Territorial Information System of Regione Veneto

The Territorial Information System for the Healthcare Organizations of the Veneto Region, referred to as SI.Ter, is the main project context of the thesis. The available documentation describes it as a regional initiative that includes application modules for territorial care and integration with existing regional and local systems. For this thesis, SI.Ter is relevant in three complementary ways. Functionally, it supports territorial services and care pathways such as

protected discharge, protected admission, home care, intermediate care, and other community-based processes. Technologically, it is described as a web-based, cloud-native, multi-layer solution based on microservices, APIs, integration middleware, workflow automation, monitoring, and DevSecOps practices. From a governance perspective, it is connected to regional project documentation, contractual requirements, milestones, and interactions among several actors. This section introduces SI.Ter as the case-study environment from which the thesis derives its analytical and technical problem: how to model and evaluate interoperability flows in a complex healthcare integration architecture.

1.1.3.1 Overview

The SI.Ter initiative concerns the realization of a Territorial Information System for the Healthcare Organizations of the Veneto Region. The project documentation presents it as part of a broader effort to support territorial healthcare services through application modules, interoperability services, and integration with existing regional platforms. The RTI technical report identifies core components such as ePersonam for COT, home care, intermediate residential structures and hospice, SisTer for services such as consultori, developmental age, neuropsychiatry and mental health, and an electronic prescription module placed transversally with respect to the other modules. The initiative is not described as a greenfield replacement of all existing systems. The documentation emphasizes continuity with solutions already in use and the need to extend and harmonize the supply across the regional territory. This is important because it clarifies the integration problem: SI.Ter must operate in an ecosystem where several systems already exist and where regional services such as APMS, Anagrafe Zero, Electronic Health Record components, XDS repositories, CUP, ADT, and other platforms remain relevant. For the thesis, SI.Ter provides the concrete domain in which interoperability flows are studied. The initiative makes visible the need to combine functional understanding of territorial care processes with technical knowledge of standards, protocols, and integration

layers. The available project documentation identifies DDR 11922/2026 as the source connected to the SI.Ter governance model. In this introductory chapter, the governance dimension is therefore limited to the project-level framing: SI.Ter is treated as a regional initiative involving Azienda Zero, the Healthcare Organizations, and a formal project governance context. [DATA MISSING: BibLaTeX entry and exact page reference for DDR 11922/2026 before using a more precise governance formulation.]

1.1.3.2 Functional Architecture

The functional architecture of SI.Ter can be understood as a set of application domains supporting territorial healthcare processes. The available documentation identifies modules related to the Centrale Operativa Territoriale, home care, intermediate care structures, hospice, consultori, developmental age services, neuropsychiatry, mental health, and electronic prescription. These components are not isolated from each other: they participate in care pathways where information must follow the patient and remain available to authorized operators. This kind of architecture is relevant to the thesis because it identifies the processes that generate interoperability requirements. Protected discharge, for example, requires the coordination of hospital-originated information, territorial evaluation, possible activation of home care or intermediate settings, and access to patient documentation. Protected admission similarly requires interaction between territorial services and clinical-care applications. These are functional scenarios before they are technical flows. At this level, the thesis does not need to describe every SI.Ter function. It focuses on the functional architecture only insofar as it explains why integration flows exist and why they involve multiple systems. The functional view provides the bridge between healthcare processes and executable workload models. The naming used here is intentionally limited to module families explicitly present in the consulted SI.Ter and RTI documentation: ePersonam-related territorial-care areas, SisTer-related services, and the transversal electronic-prescription com-

ponent. [DATA MISSING: final validation of the exact SI.Ter module names to be used consistently in the submitted thesis.]

1.1.3.3 Technological Architecture

The SI.Ter technical report describes the solution as web-based, cloud-native, modular, and organized according to a multi-layer architecture. The presentation layer includes web frontends based on modern web technologies. The interoperability layer includes WSO2 Micro Integrator, microservices for specific integration functions, RabbitMQ for asynchronous messaging, Drools for transformation and validation logic, and support for standards and formats such as REST, HL7, JSON, XML, FHIR, and XDS.b. The process automation layer uses Camunda v7 for BPMN and DMN-based workflow management. Other layers include business logic, persistence, business intelligence, DevSecOps, and monitoring. This architecture matters for the thesis because it introduces several observation and mediation points between a client request and the final application behavior. A flow may cross authentication services, gateway components, transformation logic, queues, application microservices, registries, and document repositories. As a consequence, measuring only the client-side duration of a request is not equivalent to measuring middleware processing time or backend service time. The thesis framework is not the SI.Ter platform itself. It is a separate experimental tool designed to generate and execute RVE-oriented workloads. However, its design is motivated by the technological architecture of systems like SI.Ter, where integration layers and application layers must be distinguished when interpreting measurements. For this reason, the technological stack is described at the level of component families and architectural responsibilities, without reproducing endpoint details, deployment parameters, credentials, or environment-specific configuration. [DATA MISSING: final disclosure confirmation for any more detailed description of SI.Ter technological components.]

1.1.3.4 Project Objectives and Milestones

The project documentation frames SI.Ter as a regional initiative aimed at supporting territorial healthcare services, integrating with existing systems, and aligning with the architectural requirements of Azienda Zero. The technical tender documentation states that the system must be based on a microservice logic, include application modules necessary to support the described processes, and provide modules dedicated to data integration to and from hospital information systems and Azienda Zero systems. The available RTI documentation also refers to the need to extend existing solutions, add required functionalities, preserve continuity with pre-existing systems, and define detailed schedules for development, testing, validation, and release of integrations. This shows that the objectives are both functional and architectural: support territorial care processes, integrate with the regional ecosystem, and provide a scalable and maintainable technical foundation. For this thesis, the milestone aspect is relevant only as background. The work does not evaluate project management execution or contractual delivery. Instead, it uses the project objectives to motivate a research problem: when a system must integrate many existing services and support territorial workflows, there is value in having an independent methodology for modelling and measuring integration flows before drawing conclusions about performance or capacity. Available planning notes report project milestones, but milestone dates are not necessary for the research problem developed in this chapter. If the final thesis needs a project timeline, it should be based on the validated project plan rather than on secondary notes. [DATA MISSING: official milestone table and dates, if a SI.Ter project timeline is included.]

1.1.4 Research Problem

The research problem emerges from the gap between documented interoperability requirements and controlled experimental evaluation. In the SI.Ter context, healthcare flows involve heterogeneous systems, multiple standards, and several

architectural layers. The industrial problem is the need to integrate existing applications and regional services. The technical problem is the correct orchestration of multi-step, multi-protocol flows. The experimental problem is the difficulty of generating realistic, repeatable, and observable traffic for those flows. The thesis addresses this gap by asking how healthcare interoperability flows can be translated into executable workloads that preserve relevant dependencies and can be measured without confusing framework-side observations with middleware-side evidence. This leads to four subproblems: heterogeneity among healthcare systems, complexity of multi-actor and multi-protocol flows, absence of a controlled RVE end-to-end simulation environment, and the architectural limits of directly correlating framework data with middleware data.

1.1.4.1 Interoperability among Heterogeneous Healthcare Systems

Healthcare interoperability in the SI.Ter context is heterogeneous by design. The system must interact with regional services, existing local systems, application modules, identity and assertion providers, patient registry services, Electronic Health Record components, XDS repositories, and administrative platforms. Each of these systems can expose different interfaces, data formats, security requirements, and operational constraints. The research gap is therefore concrete: it is not enough to verify that a single API endpoint can be called. A healthcare process may require several calls, each dependent on data produced by previous steps. For instance, an authentication or assertion step may enable access to a later service; a patient query may provide identifiers or context needed by a care application; a document publication flow may depend on both patient identity and document metadata. The interoperability problem is thus both semantic and temporal. This thesis treats heterogeneity as a property that must be modelled. A workload must represent flow types, transaction sequences, dependencies, inputs, outputs, and execution modes. Without such modelling, performance tests risk becoming collections of isolated requests that

do not reflect the behavior of real integration processes.

1.1.4.2 Complexity of Multi-Actor and Multi-Protocol Flows

The complexity of regional healthcare integration flows derives from the interaction of multiple actors and protocols. A single end-to-end flow can involve security services, patient registry services, context-call services, middleware, application modules, and document infrastructures. These components may communicate through SOAP, REST, XML, JSON, FHIR resources, SAML assertions, JWT tokens, or IHE profiles. The important point for the thesis is that these steps are not independent. Within a flow, later steps may require tokens, identifiers, or outputs produced by earlier steps. This creates sequential dependencies inside each flow. At the same time, a workload may contain many independent flows that can be executed concurrently. The framework therefore preserves a key invariant: steps are sequential within a flow, while concurrency is introduced across independent flows when supported by the execution engine. This distinction is central to performance evaluation. Step latency, request latency, flow completion time, throughput, and concurrency are not interchangeable. A slow authentication step can affect all dependent steps in a flow, while high concurrency can affect scheduling and resource contention across flows. The thesis uses this complexity to justify a flow-level modelling approach rather than a request-only benchmark.

1.1.4.3 Absence of a Controlled Environment for Traffic Simulation

A recurring difficulty in evaluating interoperability systems is the lack of a controlled environment that can reproduce complete application traffic while keeping inputs, schedules, and workload composition explicit. In the RVE context, this means representing not only individual transactions such as patient query or context call, but also the sequences in which those transactions occur and the data dependencies between them. The developed framework addresses this need by generating synthetic FHIR patients, composing RVE flows, building stochastic timelines, executing requests in mock or live mode,

and collecting structured JSONL logs. This does not replace validation in real environments. Rather, it provides a controlled layer where workload design, flow mix, scheduling, and logging can be inspected and repeated. The absence of such a layer would make experimental evaluation less reproducible. It would be difficult to compare configurations, isolate workload assumptions, or distinguish whether observed behavior depends on the flow model, the execution mode, the middleware, or the backend system. The framework is therefore introduced as a methodological instrument: it makes the studied flows executable and observable.

1.1.4.4 Limits of Correlation between Framework and Middleware side Observations

A key methodological constraint of the thesis is that framework-side observations and middleware-side observations cannot always be directly correlated. The framework can observe the requests it schedules, the responses it receives, the flow completion time, and the structured events it logs. Middleware components, instead, may observe routing, transformation, gateway processing, queues, retries, backend calls, or internal errors. These are related phenomena, but they are not automatically the same measurement. Direct correlation can be non-identifiable when timestamps come from different systems, clocks are not aligned, identifiers are missing, sampling differs, or the middleware aggregates events at a different level of granularity. Even when both sides report latency, the population and measurement boundaries may differ. A client-observed latency can include network time and downstream processing, while gateway latency may represent only part of the path. For this reason, the thesis adopts a two-level grey-box methodology. The first level is the framework layer, where workload generation, scheduling, request execution, and client-observed metrics are controlled. The second level is the middleware layer, where enrichment or comparison is possible only when logs contain sufficient identifiers and timing information. This prevents unjustified causal claims and supports

a more careful interpretation of performance evidence.

1.1.5 Thesis Objectives

The thesis objectives are organized around four dimensions: analytical, modelling, implementation, and measurement. The analytical objective is to study the standards, protocols, and architectures used in healthcare interoperability. The modelling objective is to formalize relevant use cases and translate them into flow-based representations. The implementation objective is to build a framework for generating, scheduling, executing, and logging RVE workloads. The measurement objective is to define metrics and an experimental method that distinguish request-level, step-level, flow-level, and time-window observations. These objectives are intentionally formulated as verifiable activities rather than as promises of final performance improvements. The thesis aims to provide a method and an artefact for evaluation; any quantitative conclusion must depend on real experimental data produced and analyzed in later chapters.

1.1.5.1 Studying Standards Protocols and Interoperability Architectures

The first objective is to study the technical foundations that make regional healthcare interoperability possible. This includes healthcare standards such as HL7 FHIR R4, IHE profiles such as XDS.b and IUA, message formats such as XML and JSON, interaction styles such as SOAP and REST, and security mechanisms based on SAML assertions and JWT tokens. The objective is not to provide an encyclopedic presentation of each standard, but to understand their role inside the specific flows considered by the thesis. This study supports the analytical pillar of the work. It makes it possible to interpret regional specifications such as Anagrafe Zero, context-call mechanisms, Affinity Domain rules, and authentication-related documentation. It also allows the thesis to distinguish between functional requirements, protocol constraints, and implementation choices. For example, FHIR is relevant for patient resources and

registry interactions, while SAML and JWT are relevant for identity propagation and access control. The expected outcome of this objective is a precise vocabulary for describing flows. Without this vocabulary, the framework would risk generating technically plausible traffic that is not connected to the semantics of the healthcare domain. The study of standards therefore prepares both the use-case formalization and the technical implementation.

1.1.5.2 Translating Internship Experience into Academic Use Cases

The second objective is to formalize the main healthcare use cases considered in the internship context, especially protected discharge and protected admission. These scenarios are relevant because they require coordination between care settings and depend on the availability of patient identity, clinical context, and application-level information. They are also representative of the broader problem of territorial care continuity. Formalization means identifying actors, systems, preconditions, information dependencies, and possible integration steps. It does not mean inventing operational details that are not documented. Where the available sources describe the involvement of COT, ADT notifications, Anagrafe Zero, Electronic Health Record consultation, XDS repositories, or context-call mechanisms, these elements can be used to motivate the flow model. Where the internship-specific activity is not documented, the thesis must mark the gap. This objective connects the healthcare domain to the framework. A use case becomes relevant for experimentation only when it can be mapped to a workload structure: flow type, ordered steps, dependencies, execution mode, and measurable outputs.

1.1.5.3 Designing and Implementing a Traffic Simulation Framework

The third objective is to design and implement a Python framework for simulating RVE application traffic end to end. The current codebase supports synthetic FHIR patient generation, flow construction, timeline generation, asynchronous execution, mock and live modes, JSONL logging, normalization, and metric computation. The framework is organized as a pipeline that moves

from patients to requests, from requests to timelines, from timelines to execution logs, and from logs to analytical reports. This objective is technical but remains linked to the domain analysis. The framework is not a generic load generator detached from healthcare semantics. Its unit of modelling is the flow, not only the isolated request. Supported flow types include sequences such as RVE-1.b followed by RVE-121 and RVE-130, RVE-1.b followed by RVE-54, RVE-1.b followed by RVE-55, and additional flows involving ITI-18, ITI-43 and ScrybaSign interactions as represented in the current codebase. The implementation objective is therefore to provide an executable artefact that can reproduce dependencies among steps, vary workload composition, schedule arrivals, and collect structured evidence for later analysis.

1.1.5.4 Defining a Measurement Methodology

The fourth objective is to define a measurement methodology based on operationally clear indicators. The framework and analytics components support metrics related to latency, throughput, error rate, concurrency, flow completion, and time-series aggregation. These metrics must be interpreted at the correct level: a request-level latency is different from step latency, flow completion time, throughput over a time window, or observed concurrency. The methodology also distinguishes between framework-observed and middleware-observed data. Framework logs can support measurements such as client-observed latency, scheduling behavior, success or failure at step level, and flow-level completion. Middleware logs, when available and joinable, can add information about gateway or upstream processing. However, the two sources must not be conflated unless identifiers, timestamps, and measurement boundaries make the comparison meaningful. This objective prepares the experimental chapters. It does not claim final results in advance; it defines the conditions under which results can later be considered valid: explicit configuration, known workload, documented execution mode, clear metric definitions, and traceable data sources.

1.1.6 Main Contributions

The thesis contributes along four lines. The analytical contribution is the technical-functional study of healthcare interoperability flows in a regional territorial-care context. The technical contribution is the implementation of a framework that can generate, schedule, execute, and log RVE-oriented workloads. The methodological contribution is a two-level grey-box approach that separates framework-side observations from middleware-side evidence. The experimental contribution is the preparation and, where data are available, production of empirical evidence for evaluating flow behavior. These contributions should be read together. The framework is not an appendix to the analysis, and the analysis is not merely background for the software. The thesis contribution lies in the connection between the two: healthcare processes are translated into executable workloads, and those workloads become the basis for controlled measurement.

1.1.6.1 Analytical: Functional Study of Interoperability Flows

The analytical contribution consists in reconstructing how healthcare interoperability flows operate within the regional territorial-care context. This includes identifying the relevant actors, systems, standards, and transaction families, and understanding how they relate to care scenarios such as protected discharge and protected admission. The analysis uses project documentation and regional specifications to connect organizational processes with technical interfaces. The originality of this contribution does not lie in redefining healthcare standards. Standards such as HL7 FHIR, IHE profiles, SAML, JWT, SOAP, REST, XML, and JSON already exist and are reused. The contribution lies in applying them to a coherent thesis problem: understanding which flows matter for territorial care, how they compose into end-to-end interactions, and what dependencies must be preserved when moving from analysis to experimentation. This analytical layer is necessary because a workload generator without domain grounding would risk testing artificial sequences. Conversely, domain

analysis without executable modelling would remain difficult to evaluate experimentally. The thesis places the analytical contribution at the foundation of the technical work.

1.1.6.2 Technical: A Framework for Generating Scheduling Executing and Measuring RVE Workloads

The technical contribution is the development of a framework that models RVE traffic as flows composed of ordered transaction steps. The framework supports a configurable flow mix, where each flow type represents a class of interaction and each weight defines its relative frequency in the generated workload. This allows experiments to represent different workload compositions without changing the implementation code. The current codebase maps flow types to transaction sequences. Examples include:

- *FLOW_CONTEXT_CALL*: the integration flow with the FSE visualizer;
- *FLOW_PATIENT_QUERY*: the integration flow with Anagrafe Zero for patient query;
- *FLOW_PATIENT_CREATE*: the integration flow with Anagrafe Zero for patient creation;
- *FLOW_ITI_18*, the integration flow with the XDS.b Registry;
- *FLOW_ITI_43*, the integration flow with the XDS.b Repository;
- *FLOW_SCRYBASIGN_SIGN*, the integration flow with the digital signature provider.

These flows are generated, scheduled over time, executed according to the selected mode, and logged as structured events. The contribution is not simply the ability to send HTTP requests. It is the combination of synthetic patient generation, dependency-aware flow construction, stochastic timeline generation, asynchronous execution, mock/live modes, placeholder resolution, JSONL logging, normalization, and metric computation. This ma-

kes the framework suitable for studying flow-level behavior rather than only endpoint-level behavior.

1.1.6.3 Methodological: A Two-Level Grey-Box Analysis Approach

The methodological contribution is the use of a two-level grey-box approach. The first level is the framework layer, where the experimenter controls workload generation, flow composition, scheduling, execution mode, and client-side logging. The second level is the middleware layer, where additional evidence may be available from gateway, routing, transformation, or upstream-processing logs. The term grey-box is used because the framework does not treat the whole system as an opaque black box, but neither does it assume full visibility into every internal component. It uses the observability available at each layer and keeps the interpretation boundaries explicit. Framework data can describe what the client generated and observed; middleware data can describe what the integration layer processed, if such data are available and joinable. This prevents an important methodological error: interpreting descriptive alignment as causal proof. Two time series may appear similar without representing the same measurement boundary. A latency increase observed at the client may depend on middleware processing, backend response time, network effects, scheduling, or workload composition. The thesis therefore uses the two-level method to support cautious design guidance rather than unsupported causal claims.

1.1.6.4 Experimental: Empirical Evidence on Performance and Robustness

The experimental contribution is the production of empirical data through controlled executions of the framework, when such executions are available. The framework is designed to generate datasets, timelines, execution logs, normalized metrics, reports, and graphs. These artefacts can support evaluation of latency distributions, throughput, error rate, concurrency, flow completion time, burst behavior, and the effects of configured fault injection. At this stage, however, the final empirical contribution remains data-gated. The current code-

base supports the generation of datasets, timelines, execution logs, normalized metrics, reports, and graphs, but this capability is not equivalent to a validated experimental result. [DATA MISSING: final executed scenarios, dataset sizes, configuration versions, logs, metric reports, and validated interpretations.] The thesis can describe the experimental capability of the framework because it is supported by the codebase, but it must not claim quantitative results until the corresponding runs have been executed and analyzed.

1.1.7 Structure of the Dissertation

The dissertation follows a narrative path from context to evaluation. After this introductory chapter, the thesis first develops the healthcare and interoperability background needed to understand the problem. It then examines the standards, protocols, and regional specifications that define the technical environment in which SI.Ter-related flows operate. The central chapters move from analysis to design. The healthcare use cases and integration flows are formalized, with particular attention to how multi-step interactions depend on authentication, patient identity, application context, and document exchange. The framework is then presented as the technical response to this modelling problem: it transforms the analyzed flows into executable workloads. The implementation chapter describes the current codebase in terms of modules, responsibilities, and verified behavior, without replacing design discussion with line-by-line code commentary. The methodology chapter defines the experimental protocol, metrics, data sources, and interpretation rules. The results chapter is intentionally data-gated: it must report only evidence derived from real runs, logs, reports, and validated analysis. Finally, the concluding chapter connects the analytical and technical outcomes, discusses limitations, and identifies future work, including possible extensions of the framework and broader experimental validation.

Domain Context and Interoperability

2.1 Introduction

This chapter develops the domain background needed to understand the interoperability problem addressed by the thesis. Chapter 1 introduced the research setting and the dual analytical and technical contribution of the work; the present chapter focuses instead on the healthcare environment in which those contributions become necessary. The reference domain is territorial healthcare in the Veneto Region, where care continuity depends on the cooperation of organizational actors, clinical-care applications, regional platforms, authentication services, patient registries, document repositories, and integration middleware. The chapter first discusses territorial care and continuity of care, with particular attention to the organizational model defined by Ministerial Decree 77/2022. It then examines the problem of information fragmentation and the need for interoperability among heterogeneous systems. Finally, it identifies the main actors and integration components involved in the regional ecosystem, and introduces the normative, organizational, security, data-management, and auditability constraints that shape the design of healthcare integration flows. The purpose is not to provide a general description of the Italian healthcare system. Instead, the chapter isolates the elements that are relevant for the thesis: why territorial processes require information continuity, why healthcare systems cannot be treated as isolated applications, and why a controlled methodology for modelling and measuring interoperability flows is required.

2.1.1 Territorial Care and Continuity

Territorial care refers to the set of healthcare and social-health services delivered close to the patient's living environment, outside or around the hospital setting. In this thesis, the concept is relevant because territorial care depends

heavily on transitions: from hospital to home care, from access points to coordination units, from general practitioners to specialist or community services, and from clinical documentation to regional information systems. Continuity of care requires that information follows these transitions in a consistent and timely way. A patient taken in charge by a territorial service may need to be identified through a regional registry, associated with a care pathway, linked to documents in the Electronic Health Record, and managed by operators with different roles and authorizations. The domain therefore creates interoperability requirements before any technical implementation is considered. This section introduces the territorial structures defined by DM 77/2022, the role of Centrali Operative Territoriali, the actors involved in patient management, and the effect of digitalization on care-taking processes.

2.1.2 Territorial Structures and Organizational Models

Ministerial Decree 77/2022 defines models and standards for the development of territorial care in the Italian National Health Service. For the thesis, the relevant point is that territorial care is organized as a network of services and coordination structures rather than as a sequence of isolated encounters. The decree identifies entities such as the District, the Casa della Comunit , the Centrale Operativa Territoriale, the 116117 operational center, home care services, community hospitals, and telemedicine-supported services. The District represents a coordination level for territorial health and social-health services. The Casa della Comunit  is conceived as a recognizable access point for citizens and as a place where multidisciplinary professionals can operate. The Centrale Operativa Territoriale, or COT, coordinates transitions across settings and supports the continuity of care. Home care and community hospitals represent further settings in which patient management depends on coordinated information exchange. The technological implications are substantial. The decree refers to systems for tracking transitions, managing and monitoring health data, accessing the Electronic Health Record, integrating with company appli-

cations, and supporting telemedicine. These requirements show that territorial care is not only an organizational reform; it also requires digital infrastructures capable of linking actors, applications, and information sources. The decree is therefore used in this chapter as the normative source for the organizational and technological rationale of territorial interoperability, especially for the district model, the COT, the 116117 operational center, home care, community hospitals, telemedicine, Electronic Health Record access, and information-system interconnection. [DATA MISSING: BibLaTeX entry and exact page references for the cited DM 77/2022 passages in the final bibliography.]

2.1.3 Role of the Centrale Operativa Territoriale

The Centrale Operativa Territoriale, abbreviated as COT, is central to the interoperability perspective of this thesis. Its function is not primarily to deliver a clinical service directly, but to coordinate the taking in charge of the person and to connect services and professionals across different care settings. In this sense, the COT is a coordination node where organizational continuity and information continuity meet. The COT needs access to updated patient information, care-transition data, and communication channels with other services. It may interact with hospital departments, home care services, general practitioners, territorial access points, and regional systems. The DM 77/2022 documentation explicitly associates COT activity with technological requirements such as tracking transitions, monitoring health data, accessing the Electronic Health Record, and integrating with company information systems. For interoperability, this means that the COT cannot be represented as a standalone application. It is part of a distributed process in which patient identity, clinical context, care assignment, and documentation must be made available across systems. Protected discharge and protected admission are therefore natural examples of COT-related processes: they involve transitions between hospital and territorial settings, require coordination among actors, and depend on reliable information exchange.

2.1.4 Other Actors and Roles

Territorial care involves several actors whose responsibilities are distinct but operationally connected. General practitioners, or *Medici di Medicina Generale* (MMG), represent a primary-care reference for patients. ULSS and other Healthcare Organizations provide organizational and operational structures for local services. Hospital departments generate or consume information related to admissions, discharges, and clinical events. *Assistenza Domiciliare Integrata* (ADI) provides integrated home care for patients requiring support at home. The relevance of these actors for the thesis lies in their information dependencies. A hospital department may trigger a discharge process that requires territorial follow-up. ADI may need patient data, care plans, clinical documentation, and coordination with the COT. MMG may need visibility over patient transitions and documents. Healthcare Organizations must manage local processes while also interoperating with regional systems and shared infrastructures. These interactions create the conditions for multi-actor flows. The information needed to manage a patient is not owned by a single system or actor. It is distributed across applications, registries, documents, and organizational responsibilities. As a result, interoperability becomes a prerequisite for continuity of care, and any experimental model of such flows must preserve the fact that several actors and systems contribute to one end-to-end process.

2.1.5 Digitalization Impact

Digitalization affects care-taking processes by making coordination more dependent on shared information systems. In territorial care, taking charge of a patient may involve patient identification, assessment, care-plan creation, assignment to a service, consultation of previous documents, production of new clinical documentation, and audit of operator actions. These steps are partly organizational and partly technical. The SI.Ter documentation describes a context in which territorial services are supported by application modules and by integration with regional or local systems. Digitalization therefore does

not simply replace paper with electronic forms. It changes the way processes are coordinated: information can be routed, transformed, validated, logged, subscribed to, or made available through regional services. At the same time, digitalization introduces new constraints, because each step must respect authorization rules, data-protection requirements, interoperability specifications, and auditability needs. For this thesis, the impact of digitalization is relevant because it turns care pathways into measurable technical flows. Once a process is mediated by services, APIs, middleware, and logs, it becomes possible to model it as a sequence of steps and to evaluate properties such as latency, completion time, error rate, and concurrency. This is the conceptual link between the healthcare domain and the framework developed in the thesis. The available SI.Ter notes support two examples that can be used at an abstract, non-operational level. In a protected-discharge pattern, hospital-originated information can trigger COT evaluation, access to patient identity and clinical documentation, and assignment to an appropriate territorial setting. In a 116117/PUA/COT pattern, a citizen-facing access channel can route a socio-health need toward the PUA and then to the COT, where the process is managed through territorial-care applications. These examples are used only as workflow patterns: organization-specific names, local configurations, operational details, and endpoint-level information remain outside the scope of this chapter. [DATA MISSING: final disclosure validation for detailed SI.Ter workflow variants, operational names, and organization-specific examples.]

2.1.6 Information Fragmentation and Interoperability

Information fragmentation occurs when data required for a care process are distributed across systems that were not originally designed as a single application. In territorial healthcare, fragmentation may concern patient identity, clinical documentation, administrative records, prescriptions, workflow states, access rights, and audit information. The problem is not only technical: it reflects the organizational distribution of healthcare responsibilities. Interope-

rability is the response to this fragmentation. It enables heterogeneous systems to exchange information, preserve meaning across boundaries, and support continuity of care. However, interoperability also introduces complexity, because each integration flow must coordinate formats, protocols, identifiers, security mechanisms, routing rules, and error handling. This section discusses fragmentation from the perspective of application heterogeneity, architectural separation among layers, middleware orchestration, and the value of interoperability for care continuity and process governance.

2.1.7 Heterogeneity of Healthcare Application Systems

Healthcare application systems are heterogeneous because they serve different purposes and have evolved under different organizational and technical constraints. In the Veneto regional context described by the SI.Ter documentation, the ecosystem includes territorial-care modules, authentication systems, patient registry services, Electronic Health Record components, XDS repositories, prescription systems, CUP systems, ADT systems, digital signature services, and administrative platforms. Each system exposes a specific operational view of the patient or the process. A registry system focuses on patient identity and demographic information. A document infrastructure focuses on clinical documents and metadata. A workflow application focuses on care coordination. An authentication service focuses on identity, roles, and access context. Interoperability has to connect these views without assuming that they share the same internal model. This heterogeneity motivates the thesis. A performance or workload evaluation that treats all requests as equivalent would miss the fact that different transactions have different semantics, dependencies, and technical constraints. The framework therefore models traffic at the level of flows and transaction sequences, not only as isolated HTTP requests.

2.1.8 Separation of Concerns

Modern healthcare platforms are usually organized into layers. The SI.Ter technical documentation describes presentation components, backend services, in-

teroperability components, workflow automation, persistence, monitoring, and external regional or local systems. This separation improves modularity and maintainability, but it also means that a user action may cross several layers before producing a completed business outcome. A frontend may initiate a request, but the relevant operation can involve backend services, middleware routing, transformation logic, regional APIs, registries, queues, or document repositories. From the user's point of view, this may appear as a single workflow step. From an interoperability perspective, it is a chain of technical interactions. This separation is important for measurement. Client-observed latency, middleware processing time, backend execution time, and regional-service response time do not describe the same boundary. The thesis therefore avoids treating a single metric as if it represented the whole system. Instead, it motivates a layered interpretation of measurements, where framework-side observations and middleware-side observations remain distinct unless reliable correlation evidence is available.

2.1.9 Orchestration Routing and Application Decoupling

When several systems participate in a healthcare process, interoperability requires orchestration, routing, transformation, and decoupling. Orchestration defines how steps are coordinated. Routing determines where a message or request must be sent. Transformation adapts formats and data representations. Decoupling reduces direct dependencies among application components, often through APIs, queues, or middleware services. The SI.Ter documentation assigns a central role to the interoperability layer, including message orchestration, support for multiple formats and protocols, asynchronous messaging, and transformation logic. These components are not auxiliary details. They are what make it possible for applications developed for different purposes to participate in a shared care process. For the thesis, orchestration also has an experimental meaning. A workload should not only contain a number of requests; it should represent the order and dependency structure of a flow. If a token must be

obtained before a context call, or if a patient query precedes a later operation, the benchmark must preserve that sequence. This is why the framework models step dependencies within flows and allows concurrency among independent flows rather than flattening all activity into unrelated requests.

2.1.10 Interoperability as Added Value

Interoperability adds value when it supports care continuity and process governance. From the care-continuity perspective, it enables information to follow the patient across settings and professionals. From the governance perspective, it makes processes more observable, traceable, and consistent with organizational rules. In territorial healthcare, both dimensions are necessary. A protected discharge process, for example, may require information from hospital systems, patient registry services, territorial coordination applications, and document infrastructures. If these systems do not interoperate, the process risks duplication, delays, manual reconciliation, or incomplete information. If they interoperate through controlled flows, the process can be more consistently represented, monitored, and audited. The thesis treats interoperability as a measurable property of process execution. It is not enough to state that systems are connected; it is also necessary to understand whether the resulting flows can be executed under defined workload conditions, whether failures are observable, and whether performance indicators can be computed at the appropriate level.

2.2 Actors and Integration Components

The interoperability ecosystem considered in the thesis includes both organizational actors and technical components. Organizational actors include Healthcare Organizations, territorial services, hospital departments, MMG, ADI, and coordination units such as the COT. Technical components include application modules, middleware, API gateways, identity providers, IAP services, patient registry systems, Electronic Health Record infrastructure, document re-

positories, context-call services, and prescription services. The purpose of this section is to identify these components only at the level needed for the thesis. Later chapters can then map them to standards, transactions, and executable flow models. This avoids turning the chapter into a catalogue of systems while still clarifying why healthcare interoperability requires careful architectural analysis.

2.2.1 Corporate and Regional Systems

Corporate and regional systems coexist in the Veneto healthcare ecosystem. Corporate systems are associated with the operations of individual Healthcare Organizations, such as local clinical applications, ADT systems, CUP systems, or document repositories. Regional systems provide shared services, such as identity infrastructure, patient registry functions, Electronic Health Record access, and regional interoperability specifications. This coexistence produces a distributed architecture of responsibility. Some information is managed locally, some is accessed through regional services, and some must be synchronized or referenced across boundaries. In such a context, a care process cannot be modelled as a single application transaction. It must be understood as an interaction among systems that differ in scope, ownership, and interface. For the thesis, this distinction matters because the framework simulates traffic that represents regional interoperability flows rather than only local application behavior. The modelled workload therefore reflects the existence of shared services and external dependencies.

2.2.2 API Gateway and Integration Middleware

API gateways and integration middleware act as mediation layers between clients, application services, and external systems. Their functions may include routing, protocol mediation, transformation, security enforcement, logging, error handling, and lifecycle management of APIs. The SI.Ter technical report describes an interoperability layer involving WSO2 Micro Integrator, micro-services, RabbitMQ, transformation logic, and API management components.

The middleware layer is particularly important for performance and observability. It can introduce processing time, retries, queues, transformations, or error-management behavior. At the same time, it can provide logs and metrics that are not visible from the client side. This dual role makes middleware both a functional enabler and a measurement boundary. The thesis therefore distinguishes framework-generated observations from middleware observations. A framework request can measure end-to-end client behavior, but middleware logs may be needed to understand gateway or upstream contributions. This distinction underlies the two-level grey-box methodology introduced in Chapter 1 and developed later in the thesis.

2.2.3 Identity Providers and Authentication Levels

Authentication and authorization are core constraints in healthcare interoperability. Operators must be identified, their roles must be known, and access to patient information must be limited to appropriate contexts. The documentation consulted for the thesis includes references to federated identity mechanisms, APMS, SAML, JWT, and Identity and Assertion Provider services. In the regional RVE flows considered by the framework, identity and assertion steps are not peripheral. They can be prerequisites for later transactions. For example, an assertion or token obtained in an earlier step may be required by a later service call. This makes security part of the execution logic of the flow rather than a separate configuration detail. From an experimental point of view, authentication-related steps affect both correctness and performance. A flow can fail if a token or assertion is missing, invalid, or not propagated correctly. A measurement framework must therefore model these dependencies explicitly, while avoiding any exposure of real credentials, certificates, or sensitive endpoint details.

2.2.4 Regional Health Registry and Anagrafe Zero

Anagrafe Zero represents a central component for patient identity and registry functions in the regional ecosystem. The consulted Anagrafe Zero specification

refers to transactions such as RVE-54 Patient Query, RVE-55 Patient ID Assignment, RVE-57 Patient Info Updating, and RVE-100 Resource Subscription. These services are connected to the management, search, creation, update, and subscription of patient identity information. For the thesis, Anagrafe Zero is relevant because patient identity is a prerequisite for many care processes. A flow involving protected discharge, protected admission, or access to clinical documents requires a reliable patient context. If patient identity is uncertain or inconsistent, later steps may become impossible or unsafe. The framework reflects this relevance by generating synthetic FHIR Patient resources and by modelling patient-related RVE flows. In particular, the current codebase includes flow types involving RVE-54 and RVE-55. This supports experiments where patient identity operations are part of a broader workload rather than isolated examples.

2.2.5 Electronic Health Record and Document Infrastructure

The Electronic Health Record and document infrastructure provide access to clinical documentation across care settings. In the regional ecosystem, XDS-based document repositories and the Affinity Domain specifications are relevant to the publication, retrieval, and indexing of documents. The SI.Ter technical documentation also refers to integration with XDS repositories and the regional Electronic Health Record. Document interoperability differs from patient-registry interoperability because it deals not only with identity, but also with clinical documents, metadata, access rules, and document lifecycle. Transactions such as ITI-18 and ITI-43 are relevant to document query and retrieval in IHE XDS.b environments, while other transactions may be involved in publication or replacement. For the thesis, the document infrastructure is part of the broader interoperability scenario. The framework currently includes ITI-18 and ITI-43 flow types, allowing document-oriented traffic to be represented in the workload model. Final claims about document-publication behavior must remain limited to what is supported by the sources and the codebase.

2.2.6 ePersonam and Context Services

The SI.Ter technical documentation identifies ePersonam as one of the application suites involved in territorial care modules such as COT, home care, intermediate care, and hospice. In the thesis, ePersonam is relevant as an example of a clinical-care application environment that may need to interact with regional services and context mechanisms. Context services are important because some flows do not simply exchange data; they enable access to an application or service while carrying identity and patient context. The RVE context-call documentation describes a pattern involving RVE-121 for obtaining a JWT token and RVE-130 for carrying the context call. In such flows, the application context depends on values produced by previous authentication or token-exchange steps. The framework models this kind of dependency through `FLOW_CONTEXT_CALL`, which includes RVE-1.b, RVE-121, and RVE-130. This makes the context-call scenario a clear example of why sequential flow modelling is necessary

2.2.7 Dematerialized Electronic Prescription

Dematerialized electronic prescription services are part of the broader healthcare integration ecosystem. The SI.Ter technical report refers to integration with the regional prescription system and mentions MEF transactions such as prescription sending and cancellation. In territorial care, prescription functions may be relevant when clinical-care applications need to interact with regional or national prescription services. For this thesis, prescription services are introduced as part of the domain context rather than as a central implemented flow. The framework focus remains on RVE and related interoperability traffic.

2.3 Normative and Organizational Constraints

Healthcare interoperability is shaped by constraints that go beyond software architecture. National and regional regulation, identity and authorization rules, health-data management requirements, document handling, traceability,

and auditability all influence how flows can be designed and evaluated. These constraints are relevant to the thesis because they explain why healthcare workloads cannot be generated as arbitrary traffic. Each flow must respect domain semantics, security requirements, and data-governance boundaries. Even in mock or simulation mode, the framework must preserve the logical structure of the process without exposing credentials, real patient data, or confidential operational details. This section summarizes the main constraint families relevant to the thesis and marks the areas where further documentary verification is required.

2.3.1 National and Regional

The national framework is represented in this thesis primarily by Ministerial Decree 77/2022, which defines models and standards for territorial care, and by the connection with Mission 6 of the National Recovery and Resilience Plan. The regional framework is represented by SI.Ter project documentation, regional interoperability specifications, and the governance role of Azienda Zero and the Veneto Healthcare Organizations. These sources impose constraints at different levels. Organizational standards define which structures and functions must exist in territorial care. Technological requirements indicate that systems must support information sharing, monitoring, Electronic Health Record access, and interconnection with existing applications. Project documentation defines architectural expectations such as microservices, API-based interoperability, integration components, and interaction with regional services. The current evidence is sufficient to identify the relevant families of constraints, but not to freeze the final legal apparatus of the thesis. The bibliography should distinguish at least between national territorial-care regulation, regional SI.Ter governance and project documentation, and technical interoperability specifications. [DATA MISSING: final list of national and regional legal references to cite in the thesis bibliography.] Until those references are finalized, this section avoids broad legal claims and remains focused on the constraints explicitly

present in the consulted documents.

2.3.2 Identity and Authorization Management

Identity and authorization constraints are central in healthcare interoperability because access to patient information must be associated with authenticated operators, roles, structures, and operational contexts. The consulted documentation refers to authentication through APMS, federated identity providers, SAML-based mechanisms, JWT tokens, and identity/assertion services. For integration flows, identity management has two implications. First, authentication is often a prerequisite for accessing later services. Second, authorization context influences which data an operator or application can access. This means that identity cannot be treated as an external detail. It is part of the flow structure. The consulted sources support a non-sensitive architectural description: APMS, federated identity providers, Identity and Assertion Provider services, SAML assertions, JWT tokens, and token-exchange steps can be discussed as mechanisms for propagating authenticated context across flows. Exact attribute sets, authorization matrices, credential material, and payload examples are not required in this chapter. [DATA MISSING: final description of the exact identity attributes and authorization rules that can be disclosed in the thesis.] The text should avoid reproducing payload examples containing personal or sensitive data and should describe only the non-sensitive architectural role of identity and authorization mechanisms.

2.3.3 Document management

Health data and clinical documents are subject to stronger constraints than ordinary application data. They must be accurate, protected, traceable, and available only to authorized actors. In the interoperability flows considered by the thesis, this affects patient registry operations, Electronic Health Record access, XDS document exchange, clinical document production, and document lifecycle management. From a technical point of view, these constraints require consistent patient identifiers, correct metadata, controlled access, and

reliable document exchange. From an experimental point of view, they imply that the framework must use synthetic patients and avoid real personal data. The goal is to preserve the structure of the flow without exposing real health information. The available documentation supports an architectural treatment of document governance through the Electronic Health Record, IHE XDS.b, Affinity Domain metadata, XDS Document Repository and Registry roles, and electronic-signature or seal mechanisms where applicable. This is sufficient to explain why document flows require controlled identifiers, metadata, access context, and lifecycle handling. [DATA MISSING: final citation for the data-protection and clinical-document governance rules to be included in the thesis.] Until then, the section remains at the architectural and methodological level.

2.3.4 Traceability and Audit

Traceability and auditability are essential in healthcare interoperability because systems must make it possible to reconstruct relevant actions, accesses, exchanges, and failures. The SI.Ter documentation refers to monitoring, logging, audit, and interoperability components capable of tracking messages and processing outcomes. IHE ATNA is also relevant in the broader healthcare interoperability domain when audit trails and node authentication are considered. For the thesis, traceability has a methodological meaning as well. The framework produces structured JSONL logs so that executions can be reconstructed and normalized for metric computation. This logging is not equivalent to a full production audit infrastructure, but it provides experimental traceability: generated flows, scheduled events, responses, failures, and completion status can be analyzed after execution. The consulted security and architecture sources support a high-level distinction between production audit infrastructure and experimental logging. Production systems may involve audit-log management and IHE ATNA-related events such as ITI-20, whereas the framework logs are designed for reconstructing experimental executions and computing metrics. [DATA MISSING: confirmation of which production audit

requirements and ATNA-related details can be cited in the final thesis.] The chapter should therefore distinguish production auditability from experimental observability.

State of the Art and Methodological Foundations

The problem addressed by this thesis lies at the intersection of two areas that must be considered jointly. On the one hand, healthcare interoperability requires integration architectures, semantic standards, and security mechanisms capable of coordinating heterogeneous systems. On the other hand, evaluating a distributed gateway requires an experimental model that correctly distinguishes latency, throughput, concurrency, reliability, and the temporal behavior of the workload. Considering only the first set of aspects would produce a static description of interfaces; focusing only on the second would reduce the workload to traffic devoid of the meaning and dependencies of healthcare processes.

This chapter therefore develops a state of the art oriented towards methodological decisions. Integration architectures and healthcare standards are examined to clarify what an executable flow must represent. Performance literature and traffic models are discussed to establish how such flows should be scheduled, observed, and aggregated. The results of the reviewed studies are not transferred directly to the regional case: studies on edge computing, MPI clusters, and load sharing have different experimental scopes and are used only to derive measurement principles, with the necessary caution. The discussion ultimately leads to implications for the framework developed in this thesis, which is presented as an instrument for translating documented interoperability flows into reproducible and measurable workloads rather than as a generic request generator.

3.1 Software Integration Architectures

In the domain considered, an integration architecture does not coincide with a single protocol or middleware product. It defines responsibilities and boundaries among service providers and consumers, mediation components, backend systems, and security mechanisms. This perspective is necessary to interpret the path of a request correctly: the same functional operation may traverse several technical layers, each exposing different timing, errors, and observable data.

This section relates service-oriented and resource-oriented paradigms to the roles of API gateways and middleware, the sequential nature of multi-step flows, and the decoupling of callers from providers. Its purpose is not to present a general taxonomy, but to isolate the properties that influence workload construction and subsequent measurement: interaction granularity, data dependencies, security enforcement points, transformations, and observation boundaries.

3.1.1 Service-Oriented Architecture and Resource-Oriented Architecture

Service-Oriented Architecture (SOA) represents capabilities in a distributed system as services exposed through explicit interfaces and contracts. The OASIS reference model deliberately avoids binding this definition to a specific technology and places the interaction between entities that provide and use capabilities in a shared context at its center C. Matthew MacKenzie et al. *Reference Model for Service Oriented Architecture 1.0*. OASIS, 12 ott. 2006. URL: <https://docs.oasis-open.org/soa-rm/v1.0/soa-rm.html> (visitato il 22/07/2026). For this thesis, the relevant aspect is not the historical association between SOA and SOAP Web Services, but the separation among the service contract, its internal implementation, and the organization that governs access. This separation allows systems developed and managed by different parties to cooperate without sharing the same internal application model.

Resource-Oriented Architecture (ROA), by contrast, adopts the resource as its primary conceptual unit, identified and manipulated through uniform operations. In healthcare, HL7 Fast Healthcare Interoperability Resources (FHIR) expresses this approach clearly: information entities such as patients are represented as resources, while RESTful interactions apply HTTP operations within the constraints defined by the specification and adopted profiles HL7 International. *FHIR Release 4 (Version 4.0.1): Overview*. URL: <https://hl7.org/fhir/R4/overview.html> (visitato il 22/07/2026). ROA does not remove the contract; it distributes it across the resource model, profiles, permitted operations, search parameters, code systems, and conformance rules.

SOA and ROA should therefore not be treated as mutually exclusive alternatives. A healthcare platform may expose services with clearly delimited functional responsibilities and use FHIR resources and REST interactions within them. Similarly, the same architecture may include SOAP document services conforming to IHE profiles and REST patient-registry services based on FHIR. The useful distinction concerns two ways of defining contract granularity: the first emphasizes the provided capability and operation, whereas the second emphasizes resource representation and lifecycle.

This coexistence is documented in the regional case. The Anagrafe Zero specifications describe patient-registry transactions built on FHIR R4, including patient search and identifier assignment, while XDS.b document integration uses IHE actors and transactions with distinct responsibilities Elena Vio et al. *Specifiche tecniche – Anagrafe Zero*. Ver. 2.6. Consorzio Arsenal.IT, 9 gen. 2024; IHE International. *Cross-Enterprise Document Sharing (XDS.b)*. URL: <https://profiles.ihe.net/ITI/TF/Volume1/ch-10.html> (visitato il 22/07/2026); Azienda ULSS 3 Serenissima. *Specifiche tecniche di integrazione tra sistemi Document Source e Repository XValue*. Ver. 2.9. 5 Ago. 2024. The architectural implication is that the workload cannot assume a single interaction style or serialization. It must preserve the actual contract of each step while representing the process as an end-to-end unit.

From a performance perspective, the selected granularity affects the number of round trips, message size, connection reuse, and the points at which transformations or controls may be applied. This does not imply that one style is inherently faster than another: performance depends on the specific profile, payload size, security mechanisms, and infrastructure path. The reviewed literature shows that application-perceived latency is multidimensional and that apparently local choices, such as connection management and serialization, may alter the distribution of completion times Marco Iorio, Fulvio Risso e Claudio Casetti. «When Latency Matters: Measurements and Lessons Learned». In: *ACM SIGCOMM Computer Communication Review* 51.4 (ott. 2021).

The methodological consequence for the framework is twofold. First, the transaction type must remain an explicit attribute to support consistent aggregation and comparison among homogeneous operations. Second, the unit of work cannot be reduced to an HTTP operation: a healthcare capability may require authentication, patient-registry queries, context acquisition, and document access. SOA and ROA therefore provide the architectural vocabulary for describing individual contracts, while the flow model provides the unit required to evaluate their composition.

3.1.2 API Gateways, Middleware, and Orchestration Patterns

An API gateway provides a controlled entry point to a set of services. Depending on the specific architecture, it may apply routing, credential or token verification, access policies, traffic limiting, and event collection. Integration middleware may have a broader scope: in addition to exposing APIs, it may mediate protocols and formats, transform messages, coordinate calls, and connect synchronous services with asynchronous components. These functions may be distributed across separate components or combined in the same product; for analysis, distinguishing them by responsibility is more useful than relying on product terminology.

Routing selects the destination or path of a request. Mediation adapts the

contract between caller and provider, for example by translating an external interface into a call to a regional system. Transformation modifies the representation or structure of a message when the two sides do not use the same format. Security verifies identity, tokens, channel, and authorization context. Observability produces the events required to reconstruct the path and outcome. These responsibilities are not interchangeable: a correctly routed request may be rejected for authorization reasons, while a valid backend response may be altered or hidden by a transformation error.

Orchestration adds a temporal dimension. An orchestrator establishes the order of operations, evaluates preconditions, and propagates data between steps. In the flows examined in this thesis, a value obtained from a security service may be required to invoke the next service, and an identifier returned by a query may become the input of a subsequent operation. Orchestration must not be confused with simply concatenating URLs: it preserves the logical state of a flow occurrence even when the individual services are stateless.

These responsibilities determine two measurement boundaries. The first is observable by the traffic generator and includes the time between sending and receiving each request and the total flow duration. The second lies inside the middleware, where gateway time, waits, upstream calls, retries, or infrastructure data may be observable. The availability of the second level depends on the actual presence of correlatable logs and cannot be inferred from client time alone. The grey-box model adopted by the thesis follows precisely from this distinction: it integrates observations from different levels without declaring them equivalent.

The gateway is also a point at which offered load may differ from the load actually forwarded. Authorization checks, concurrency limits, timeouts, or circuit breakers may prevent a request from reaching the backend. Client-side throughput, the number of requests accepted by the gateway, and the number of operations completed by the backend therefore answer different questions. Any aggregation must state the boundary and observed population explicitly.

In the framework, these principles are reflected in the preservation of flow, occurrence, request, and step identifiers. These attributes enable joins between framework events and middleware data when both are available. On their own, however, they do not demonstrate a causal decomposition of latency: correlation requires compatible keys, clocks, and sufficiently reliable event semantics. The experimental methodology must therefore treat correlation coverage and quality as dataset properties rather than implicit assumptions.

3.1.3 Synchronous Communication, Stateless Services, and Multi-Step Flows

Synchronous communication ties caller progress to the availability of a response, an error, or a timeout. This property simplifies the semantics of some transactions, but makes the duration of each step sensitive to the network, security mechanisms, middleware, and remote-system processing. In a multi-step flow, this dependency propagates: a subsequent step cannot begin when it requires an output that the preceding step has not yet produced.

The stateless nature of a service does not contradict this sequencing. Stateless means that each request contains, or makes it possible to reconstruct, the context required by the service to process it; it does not mean that the application process has no state. The flow coordinator must preserve identifiers, tokens, assertions, or results required for continuation. State therefore moves from the individual service to the application composition or exchanged data.

Regional specifications make this distinction concrete. The context call describes a transaction that obtains a JWT from a SAML assertion and the information required to access services; the resulting token is intended for subsequent interactions Gregorio Canal e Gianluca Pavan. *Specifiche tecniche chiamate di contesto RVE*. ver. 1.3. Consorzio Arsenal.IT, 18 lug. 2025. Anagrafe Zero separates patient search from identity assignment or modification and, under specified circumstances, requires a preliminary search Vio et al., *Specifiche tecniche – Anagrafe Zero*. XDS.b similarly distinguishes metadata queries

from document retrieval IHE International, *Cross-Enterprise Document Sharing (XDS.b)*. Security, registry, context, and document services therefore form an information chain rather than a collection of independent requests.

For a single flow occurrence, ordering is part of correctness. Advancing a call or reusing an output that belongs to another occurrence would produce a technically executable but semantically incorrect workload. Independent occurrences, by contrast, may overlap in time. The concurrency relevant to this thesis is therefore concurrency among flows, while step order and interruption after a blocking failure remain valid within each flow.

This structure also changes metric interpretation. Step latency measures one interaction observed by the client; flow completion time measures the interval required to complete the entire sequence. The sum of step latencies may differ from flow duration when local processing, dependency resolution, or other intervals are present. Moreover, an early error may prevent later steps from running: counting those later steps as successes, errors, or missing requests without stating the rule would alter the interpretation of the experiment.

The framework implements these semantics through ordered steps, placeholders resolved at runtime in live mode, and a separate context for each occurrence. A concurrency limit encloses the entire flow, while multiple flows are scheduled as asynchronous tasks. In this chapter, these details verify that the model is implementable; the code-level description is deferred to the design and implementation chapters.

3.1.4 Decoupling Callers from Service Providers

Decoupling reduces the knowledge that a caller must have about the implementation and location of a service provider. The caller depends on a stable contract and access point, while routing, instance selection, transformations, and policies may evolve within the integration layer. In healthcare, where regional services, Healthcare Organization applications, and document repositories fall under different organizational responsibilities, this separation is both

a maintainability requirement and a technical pattern.

Decoupling may be spatial, temporal, or representational. It is spatial when the caller does not know the physical instance that will process the request; temporal when producer and consumer need not be available simultaneously, as in queue-based systems; and representational when middleware mediates formats or contracts. The synchronous flows examined in this thesis primarily implement the first and third forms. Any asynchronous components in the broader ecosystem must not be attributed automatically to the modelled transactions.

This advantage has an observability cost. An additional layer introduces possible waits, transformations, controls, and failure modes. Client latency includes the overall result of the path but does not identify which component produced the delay. A middleware measurement may isolate part of the path, while still depending on logging quality and the availability of correlation keys. Architectural decoupling therefore requires an equally explicit observability strategy.

Distinguishing mediation, routing, security, transformation, and observability prevents over-interpretation. If the gateway rejects a token, the backend may never be involved; if a transformation fails, the observed time does not measure remote-service capacity; if a timeout occurs after forwarding, the client and middleware may record different outcomes. The experiment must preserve these cases rather than flattening them into a single error category.

Decoupling is also relevant when comparing direct and mediated paths. A valid comparison requires the two workloads to represent semantically comparable operations and the path difference to be a controlled variable. Without this control, attributing the difference between latency distributions to the middleware would be an unsupported causal inference. The framework provides path categories and optional middleware data, but the results chapter may draw conclusions only from scenarios that were actually executed and documented.

3.2 Healthcare Standards and Protocols

Healthcare standards operate at different levels. FHIR defines resources and interactions; Integrating the Healthcare Enterprise (IHE) profiles coordinate actors and transactions for specific interoperability problems; regional specifications constrain and contextualize these standards through profiles, code systems, and operational rules. Protocols, formats, and tokens complete the technical contract but do not replace healthcare semantics.

This distinction is essential to the thesis. The workload must reproduce the form of the interactions actually studied without assuming that conformance to a general standard automatically entails conformance to a regional profile. The following subsections therefore examine FHIR R4, IUA, XDS.b, document transactions, and security mechanisms only in relation to the aspects that influence flow composition and measurement. Fields, cardinalities, and code systems not required for the methodological argument remain delegated to the primary specifications.

3.2.1 HL7 FHIR R4

HL7 FHIR R4 organizes healthcare information into resources with defined structure and semantics, which can be linked through references and exchanged according to different paradigms. Published version 4.0.1 of the R4 specification includes a RESTful API, messaging, documents, and operations; it therefore does not coincide solely with the use of JSON over HTTP HL7 International, *FHIR Release 4 (Version 4.0.1): Overview*. A resource represents an interoperable concept, while profiles and extensions constrain or extend its use for a specific context.

The `Patient` resource is central to the registry case, but its concrete meaning depends on the regional profile. Anagrafe Zero v2.6 states its alignment with FHIR R4 and defines its own transactions, actors, and rules for regional patient identity Vio et al., *Specifiche tecniche – Anagrafe Zero*. Syntactic conformance of a generic Patient resource is therefore insufficient: identifiers,

extensions, code systems, and constraints must be evaluated against the applicable specification. In the framework, the synthetic generator adopts the structure required by the regional case; this chapter does not reproduce cardinality tables, for which the technical document remains authoritative.

The **Bundle** resource is a container for a collection of resources and supports different uses, including search results, messages, documents, and atomic transactions HL7 International. *FHIR Release 4 (Version 4.0.1): Resource Bundle*. URL: <https://hl7.org/fhir/R4/bundle.html> (visitato il 22/07/2026). Its type is therefore semantically significant. Within the scope examined, RVE-54 returns a collection of registry results represented as a Bundle containing Patient entries, whereas the RVE-55 request uses a **transaction** Bundle containing the Patient resource to register; the response in turn contains the outcome and resulting resource according to the regional profile Vio et al., *Specifiche tecniche – Anagrafe Zero*. These differences prevent all Bundles from being treated as equivalent payloads.

The current framework represents Patient and Bundle explicitly. For the query flow, it generates parameters from the synthetic resource and, in mock mode, provides a **searchset** response Bundle; for identifier assignment, it builds an XML-serialized **transaction** Bundle. These choices are implementation evidence, not a general statement about the mandatory format of all FHIR services. The regional specification remains authoritative for RVE messages; the code establishes only what the framework can currently produce and execute.

FHIR also contributes to workload measurability. Resource type, Bundle type, and operation affect payload size, serialization complexity, and outcome semantics. Latency comparisons among heterogeneous requests must therefore be stratified at least by transaction and, where necessary, by size. Structured resources make it possible to generate coherent synthetic data, but do not automatically make the traffic mix realistic: the flow distribution remains a separate experimental variable.

3.2.2 IHE IUA – Internet User Authorization

The IHE Internet User Authorization (IUA) profile adds authorization information to RESTful HTTP transactions. In the official specification, its principal actors are the Authorization Client, Authorization Server, and Resource Server. The client obtains an access token from the Authorization Server and incorporates it into the request; the Resource Server validates the token and applies the access decision, possibly enforcing additional service-specific policies IHE International. *Internet User Authorization (IUA)*. URL: <https://profiles.ihe.net/ITI/IUA/> (visitato il 22/07/2026).

This separation clarifies a distinction that is often blurred: authentication and authorization are not synonyms. Authentication establishes or verifies an identity; the Authorization Server uses identity, client, requested resource, and other information to issue a token; and the Resource Server ultimately applies policy at access time. IUA does not prescribe a single user-authentication method, because that choice depends on the project or national extension. Instead, the profile defines how the authorization result participates in RESTful interactions.

The current IUA specification describes the Get Access Token, Incorporate Access Token, Introspect Token, and Get Authorization Server Metadata transactions, with options for JWT, SAML, or token introspection *ibid*. The regional case examined here must not be equated automatically with every detail of the current profile release. The Anagrafe Zero specifications refer to IUA in their security considerations, while the regional security and context-call documents define RVE transactions and their own procedures for producing and using assertions and tokens Mauro Zanardini et al. *Infrastruttura di sicurezza GDL-O Sicurezza*. Ver. 2.14. Consorzio Arsenal.IT, 28 giu. 2024; Canal e Pavan, *Specifiche tecniche chiamata di contesto RVE*. The relationship is therefore one of profiling and contextualization, not nominal equivalence.

IUA is relevant to this thesis because it introduces steps that belong to both the functional path and its time budget. Token acquisition may precede the

protected call; validation may occur at the gateway or Resource Server; and a rejection may interrupt the flow before the healthcare service is invoked. A generator that sent only the final request would omit a significant part of the end-to-end behavior.

From an experimental perspective, tokens and assertions must be treated as dependencies, not as data to expose in the results. Logs must record transaction type, outcome, and timing without including credentials or sensitive contents. In mock mode, synthetic values may be propagated; in live mode, resolution must occur within each individual flow occurrence. This preserves the authorization structure while keeping secret management separate.

3.2.3 IHE XDS.b – Cross-Enterprise Document Sharing

The IHE Cross-Enterprise Document Sharing (XDS.b) profile organizes the sharing of clinical documents among organizations participating in an Affinity Domain. The domain shares the policies and conventions required to make documents searchable and retrievable, while the profile separates content persistence from metadata indexing IHE International, *Cross-Enterprise Document Sharing (XDS.b)*; CNR (ICAR) e Dipartimento per la Trasformazione Digitale. *Specifiche tecniche per l'interoperabilità tra i sistemi regionali di FSE: Affinity Domain Italia*. Ver. 2.6.3. 31 Ott. 2025.

The Document Source produces and publishes the document together with the required metadata. The Document Repository receives and persistently stores the content and registers its metadata with the Document Registry. The Registry maintains Document Entries and responds to queries from the Document Consumer. The Consumer searches the Registry using defined criteria and retrieves the document from the Repository identified in its metadata IHE International, *Cross-Enterprise Document Sharing (XDS.b)*. This separation is fundamental: the Registry is not the document store, and the Repository does not replace the metadata-query service.

The consulted XValue integration specification confirms the same division

of responsibilities in the regional context and documents publication through ITI-41 together with the metadata required for indexing Azienda ULSS 3 Serenissima, *Specifiche tecniche di integrazione tra sistemi Document Source e Repository XValue*. The Affinity Domain Italia v2.6.3 document defines constraints and value sets for metadata such as XDSDocumentEntry attributes and submission structures CNR (ICAR) e Dipartimento per la Trasformazione Digitale, *Specifiche tecniche per l'interoperabilit  tra i sistemi regionali di FSE: Affinity Domain Italia*. These tables are not reproduced here: the purpose is to show that metadata form part of the interoperable contract rather than being ancillary attributes.

XDS.b is content-neutral with respect to the document: the profile governs publication, registration, query, and retrieval without requiring the clinical content to have one representation. Semantic interoperability therefore also depends on Affinity Domain policies concerning format, structure, code systems, and classifications. For workload construction, this means that document size and type may vary independently of transaction form and must be controlled when they affect transfer times.

Security applies across the actors without changing their document-related responsibilities. The regional XValue specification requires a prior security assertion for the described transactions Azienda ULSS 3 Serenissima, *Specifiche tecniche di integrazione tra sistemi Document Source e Repository XValue*; this chapter nevertheless keeps the explanation of XDS.b separate from that of SAML and JWT. Experimentally, a document flow must preserve its security dependency and distinguish metadata query from content retrieval. Only then can end-to-end time be interpreted with respect to the steps actually executed.

3.2.4 ITI Transactions for Document Publication, Query, and Retrieval

ITI transactions operationalize the XDS.b model by assigning precise responsibilities to actor pairs. Publication uses Provide and Register Document

Set-b [ITI-41], initiated by the Document Source towards the Document Repository. The request includes both the document and its metadata; the Repository stores the content and forwards the metadata to the Registry through Register Document Set-b [ITI-42] IHE International, *Cross-Enterprise Document Sharing (XDS.b)*. In the regional XValue context, ITI-41 is explicitly identified as the transaction used to publish the document and transmit its associated metadata Azienda ULSS 3 Serenissima, *Specifiche tecniche di integrazione tra sistemi Document Source e Repository XValue*.

Queries use Registry Stored Query [ITI-18]. The Document Consumer queries the Document Registry; the Registry applies the stored-query criteria and returns metadata for matching Document Entries, including the elements required to locate the document and the Repository that stores it IHE International, *Cross-Enterprise Document Sharing (XDS.b)*. The query result is therefore not the clinical document itself. Conflating ITI-18 with retrieval would remove a distinct step from the model and alter both its semantics and its measurement.

Retrieval uses Retrieve Document Set [ITI-43]. After selecting a Document Entry, the Document Consumer requests the corresponding content from the Document Repository. This transaction may transfer payloads substantially larger than a query response; latency and throughput are therefore not directly comparable without considering document size and type. The logical sequence is query, selection, and retrieval, although individual steps may still be executed in controlled experimental scenarios.

The current framework includes flows for ITI-18 and ITI-43, each preceded by acquisition of an RVE-1.b assertion. ITI-41 is documented among the transactions relevant to the architecture but is not currently a flow type in the generator. This difference between analytical scope and implementation coverage must remain explicit: the chapter may describe ITI-41 as a foundation of the document process, but must not attribute execution of an unimplemented transaction to the framework.

The three transactions also suggest distinct metric populations. ITI-18 pro-

duces query observations, ITI-43 retrieval observations, and ITI-41 publication observations. Combining them into one mean would erase differences in responsibility and payload. The methodology must therefore stratify by transaction type and, for document operations, treat message size as a controlled variable.

3.2.5 SOAP, REST, XML, JSON, and Interoperability Profiles

SOAP and REST belong to different conceptual levels: SOAP defines an envelope-based messaging protocol, whereas REST is an architectural style that, in the healthcare APIs considered here, is implemented through HTTP semantics and resource identification. XML and JSON are representation formats and do not determine the interaction style on their own. Their concrete combination is established by the applicable standard and profile.

The documentary corpus uses SOAP/XML for security and XDS.b transactions. Anagrafe Zero instead uses FHIR-based HTTP interactions, while the RVE context call uses HTTP with JSON payloads Zanardini et al., *Infrastruttura di sicurezza GDL-O Sicurezza*; Vio et al., *Specifiche tecniche – Anagrafe Zero*; Canal e Pavan, *Specifiche tecniche chiamata di contesto RVE*; Azienda ULSS 3 Serenissima, *Specifiche tecniche di integrazione tra sistemi Document Source e Repository XValue*. It is therefore inaccurate to describe the ecosystem as a linear migration from SOAP to REST. The two paradigms coexist because they implement different contracts and profiles.

For evaluation, the relevant aspects are the number of exchanges, payload size, serialization and parsing costs, connection management, and security mechanisms. In their task-offloading scenario, Iorio, Risso, and Casetti show that application protocol, connection persistence, and serialization may affect flow completion time; they also stress that the effect depends on the messages and infrastructure under examination Iorio, Risso e Casetti, *«When Latency Matters: Measurements and Lessons Learned»*. These results do not justify classifying XML as necessarily slow or JSON as necessarily efficient in RVE flows.

The framework must therefore reproduce the format required by each transaction and record payload size. A comparison between SOAP/XML and REST/JSON would be meaningful only if the operations were functionally comparable and other variables controlled; in the current case, the formats often identify different services. Analysis by transaction type, with size possibly used as a descriptive covariate rather than a demonstrated cause, is more appropriate.

Error handling also differs among profiles. HTTP status codes, SOAP faults, FHIR errors, and timeouts must be normalized without losing their original information. The normalized layer supports common aggregation, while transaction type and original status preserve the possibility of profile-specific analysis. This dual representation avoids both complete fragmentation of the metrics and the flattening of non-equivalent semantics.

3.2.6 SAML, JWT, and Token-Based Authorization

Security Assertion Markup Language (SAML) and JSON Web Token (JWT) are structured formats that may carry assertions and claims, but their use must be interpreted within the protocol and profile that produce and consume them. A SAML assertion is not synonymous with login, just as the presence of a JWT does not by itself demonstrate authorization for a specific operation. The decision depends on issuer, audience, temporal validity, context, and enforced policies.

In the documented regional infrastructure, the RVE-1.b transaction involves an Identity and Assertion Provider and produces a SAML assertion for subsequent interactions Zanardini et al., *Infrastruttura di sicurezza GDL-O Sicurezza*. The context-call specification describes RVE-121 GetAccessToken: an AccessTokenRequester sends a request to the AccessTokenManager containing the assertion and context information; after verification, the manager returns a JWT that can be used to access the requested services Canal e Pavan, *Specifiche tecniche chiamate di contesto RVE*. This step is a contextualized token exchange, not a purely syntactic conversion.

RVE-121 precedes RVE-130 in the implemented context-call flow. The dependency is both temporal and informational because the final call requires the output obtained previously. Token validity also introduces a constraint on total duration and reuse. This chapter does not report unnecessary lifetime values or specific claims; such details belong to particular specification versions and environment configurations.

For a different application scope, the consulted APMS specifications describe OpenID Connect flows involving tokens and controls applied by the API gateway *Specifiche di integrazione autenticazione APMS*. ver. 1.4.1. Documento tecnico di progetto consultato nel corpus documentale. This material shows that different identity and authorization mechanisms may coexist in the same ecosystem. APMS, IUA, and RVE transactions must not be merged into one universal flow: each document defines its own actors, audiences, and purposes.

For workload construction, the implication is that tokens and assertions are typed outputs correlated with a flow occurrence. In live mode, they must be extracted from the response and inserted only into dependent steps; in mock mode, they may be synthetic while preserving the same dependency structure. Logs must not retain their contents. Metrics should instead expose security-step latency, errors that interrupt the flow, and their effect on completion time, without inferring the organizational cause of denial from the status code alone.

3.3 Performance Measurement in Distributed Systems

The performance of a distributed system cannot be represented by a single number. A request traverses the network, transport, security, middleware, and application service; a healthcare flow combines several requests and may terminate before completion. Latency, throughput, errors, and concurrency therefore describe distinct aspects and must state their observation boundary, population, unit, and time window.

The selected literature highlights two complementary risks. Point-to-point measurements may be too narrow to predict application behavior, while global

means may conceal long tails and temporal variation. The following subsections therefore introduce application-level flow completion time, the limitations of elementary synthetic benchmarks, the interaction between load and capacity, and the role of percentiles and time series. These foundations prepare the operational definitions of the experimental protocol without anticipating values or results that have not yet been produced.

3.3.1 Network Latency, Transport Latency, and Application-Level Flow Completion Time

Point-to-point network latency estimates the time required for a packet or probe to travel between two nodes and return to the sender. The round-trip time (RTT) obtained with tools such as ping mainly reflects propagation, forwarding, and queuing as observed by the probe protocol. It does not necessarily include connection establishment, security, complete payload transmission, or application processing.

Transport latency extends the boundary to connection and protocol behavior. Handshakes, congestion control, retransmissions, delayed acknowledgments, and connection reuse may alter the time exposed to the application. Iorio, Risso, and Casetti explicitly distinguish network RTT, TCP-level RTT, and application-level flow completion time (FCT), observing that the latter includes message transmission, middleboxes, and application-infrastructure behavior Iorio, Risso e Casetti, «[When Latency Matters: Measurements and Lessons Learned](#)».

In this thesis, *request latency* denotes the time observed by the framework between the beginning of a step request and the availability of its response or error. It is a client-side application measurement, not a pure network measurement. *Flow completion time* instead denotes the interval between the start and completion of an entire multi-step occurrence. This duration includes the steps actually executed and intermediate framework processing, and must therefore not be called RTT.

The two levels answer different questions. Request latency supports comparisons among homogeneous transactions and helps identify which step contributes to duration. Flow completion time more directly represents the time required to obtain the outcome of the simulated process. A failed flow must be analyzed separately or accompanied by the step at which it stopped, because its duration is not that of a successful completion.

When a correlatable middleware log is available, internal gateway and upstream measurements may be added. They do not replace the client measurement and cannot be reconstructed reliably by subtracting unsynchronized values. Decomposition is valid only for events that provide compatible data and a verified join key.

Finally, every latency distribution must state its population: requests, steps of a specific type, completed flows, or observations within a time window. Mixing levels produces formally computable but semantically meaningless percentiles. This distinction underpins the later separation among request-level, step-level, flow-level, and time-window metrics.

3.3.2 Limitations of Point-to-Point Measurements and Ping-Pong Benchmarks

Point-to-point benchmarks are useful for characterizing a link or communication implementation under controlled conditions. A ping-pong test measures repeated exchanges between two endpoints and may isolate RTT and bandwidth for selected message sizes. Simplicity is an advantage: the test is repeatable, permits focused comparisons, and reduces the number of variables. Its limitation appears when the result is treated as a sufficient prediction of application performance.

Dimitrov and Skjellum compare polling and blocking completion modes in MPI clusters and show that a marked difference in point-to-point latency does not necessarily produce a corresponding difference in the application benchmarks considered Rossen Dimitrov e Anthony Skjellum. «Impact of Latency on

Applications' Performance». Manoscritto consultato nel corpus documentale. The examined applications primarily exchange medium or large messages and are more sensitive to sustained bandwidth and communication patterns. This result does not concern healthcare gateways, but demonstrates that transferring conclusions from a microbenchmark to an application requires a workload model.

In the RVE case, a ping to an endpoint does not include authentication, SOAP or FHIR parsing, gateway routing, repository access, application processing, or dependencies among steps. Even a single-request HTTP benchmark remains incomplete when the actual process requires tokens or data produced by preceding calls. It may nevertheless be useful as a baseline for isolating connectivity problems or characterizing one transaction.

The limitation is therefore not an absolute defect of the instrument, but a matter of external validity. A proportionate mitigation is to combine three levels: elementary environment measurements when available, application-step latency, and flow completion time. Comparisons among levels must remain descriptive; an observed correlation does not automatically identify a cause.

A second limitation concerns temporal representativeness. Short serial tests do not reproduce concurrency, queues, bursts, or routing variations. A microbenchmark may expose the minimum achievable under favorable conditions, but not the distribution under mixed load. For this reason, the framework schedules multiple occurrences and preserves windowed time series, while the experimental protocol must specify seeds, duration, and repetitions.

The resulting principle is that each benchmark must be evaluated against the research question. If the question concerns the minimum cost of one call, a point-to-point measurement may be appropriate. If it concerns the gateway's ability to sustain composed healthcare flows, an end-to-end measurement with a semantically valid workload is required.

3.3.3 Throughput, Errors, Concurrency, and Execution Queues

Offered load is the rate at which the generator schedules new occurrences; throughput is the observed completion rate. If $N_c(W)$ is the number of units completed within a window W of duration $|W|$, throughput is

$$X(W) = \frac{N_c(W)}{|W|}. \quad (3.1)$$

The unit must identify the counted object: requests per second or flows per second. The framework computes both forms globally and, where applicable, by flow type. A global value over the whole run is a summary, whereas a series of time buckets exposes variation and transients.

The error rate is the ratio of observations classified as errors to the population considered. The denominator is again part of the definition: errors per request, per transaction, and incomplete flows are not equivalent. An error in an initial step may prevent subsequent steps from running; the flow success rate captures the outcome of the entire occurrence, while the distribution of failure steps describes error propagation.

Concurrency measures the number of tasks active simultaneously, not the total number scheduled. The framework observes active flows and active steps. Because steps within one occurrence are sequential, an increase in active steps results from overlap among independent flows. The configured limit determines how many occurrences may enter the execution section; it does not necessarily equal the throughput achieved.

When offered load temporarily exceeds available capacity, scheduled events may start late. The framework records *queue delay*, or scheduling drift, as the difference between actual and planned start times. This is a generator-side measurement: it exposes pressure on the scheduler or concurrency limit, but does not represent an internal gateway queue. Middleware queues require their own observations.

The metrics interact. In a stable regime, throughput and offered load may

rise together. As capacity is approached, concurrency and delay may increase; beyond a certain regime, timeouts or errors may appear while useful throughput plateaus or declines. This is an interpretive structure to be tested against data, not a trend that can be attributed to the thesis experiments in advance.

Chapter 8 must therefore state the formula, unit, level, and source of every KPI. Examples include request throughput in requests/s from completed events; flow throughput in flows/s from `flow_completed` events; error rate as a percentage of the request population; flow success rate as a percentage of occurrences; concurrency as active flows or steps sampled at send time; and queue delay in milliseconds. Results may be reported only for scenarios that were actually executed.

3.3.4 Percentiles, Long Tails, and Temporal Stability of Measurements

The mean describes the sample expectation but does not show how observations are distributed. Distributed-system latencies are often asymmetric and long-tailed: a small number of high values may be operationally important while having limited effect on the median. Percentiles address this need. Percentile p is the value below which a fraction $p/100$ of the observed population falls.

P50 describes median behavior, while P95 and P99 characterize progressively more extreme portions of the distribution. They are not interchangeable with the maximum, which depends strongly on sample size, and do not constitute guarantees for individual observations. Iorio, Risso, and Casetti highlight the difficulty of obtaining stable estimates beyond P99 and recommend repeated and prolonged observations to reduce the influence of occasional events Iorio, Risso e Casetti, «[When Latency Matters: Measurements and Lessons Learned](#)».

A percentile is interpretable only when its population and window are stated. A global P99 over all requests mixes different transactions; P99 by tran-

saction type permits a more consistent comparison. P95 of flow completion time concerns flows rather than steps. A percentile computed per time bucket also describes a different phenomenon from one computed over the entire run.

Temporal stability requires both time series and repetitions. An aggregate distribution may conceal an interval of degradation offset by regular periods. Buckets of declared duration make it possible to observe throughput, errors, concurrency, queue delay, and latency over time. Repetitions with controlled seeds help separate synthetic-load variability from system variability, although they do not remove variability in a live environment.

The framework’s descriptive statistics include the median and percentiles through P99, and the normalized dataset produces configurable time buckets. This technical availability does not replace experimental design: extreme-tail analysis requires sufficient samples, and the sample size must be reported. When the population is small, a high percentile should be presented cautiously or omitted.

3.3.5 Effects of Access Networks, Routing, Buffering, and Application Protocols

Application time results from contributions distributed across the entire path. Access networks introduce different technologies, capacities, and sharing conditions; routing determines paths that do not depend solely on geographical distance; buffering can absorb bursts but may also create delay under congestion; and transport, security, and application protocols add handshakes, congestion control, encoding, and parsing.

The measurement campaign by Iorio, Risso, and Casetti varies access networks, server placement, message size, protocol, and connection management. Its principal conclusion is that moving the data center closer reduces one delay component but does not by itself guarantee low flow completion time: transmission, queuing, transport, application, and orchestration may dominate the path [ibid.](#)

For a healthcare gateway, middleware introduces further functions, including token verification, routing, transformation, and upstream forwarding. These operations belong to the application layer and cannot be inferred from network RTT. Similarly, high latency does not demonstrate middleware saturation: the cause may lie in the access network, the regional service, document size, or a combination of factors.

The grey-box methodology addresses this indeterminacy by separating the client boundary from the middleware boundary. The first reports the generator’s experience and flow duration; the second, when available, adds internal components and infrastructure data. Interpretation must avoid unsupported subtraction and correlation. Even when CPU or connection data are available, establishing causality requires a controlled experiment rather than statistical correlation alone.

3.4 Workload and Traffic Models

Performance evaluation requires not only valid requests, but also an explicit rule defining when they arrive at the system and in what composition. The load model separates workload content from its temporal dynamics: the flow mix describes which flows are selected; the arrival process determines their timestamps; and bursts and non-stationary profiles introduce controlled transients.

In the framework, the timeline is constructed from previously generated requests and does not create new healthcare semantics. This boundary makes it possible to vary the temporal profile without altering flow contracts. The following subsections define the Poisson process, variable-rate profiles, burst injection, and the meaning of a synthetic workload. Parameter values belong to experimental scenarios and are not presented as representative of real healthcare traffic in the absence of observational data.

3.4.1 Stochastic Arrival Processes and the Poisson Distribution

A homogeneous Poisson process models independent arrivals with a constant rate $\lambda > 0$. The number of arrivals $N(t)$ in an interval of duration t follows

$$\Pr[N(t) = k] = \frac{(\lambda t)^k e^{-\lambda t}}{k!}, \quad (3.2)$$

while inter-arrival times are exponentially distributed with parameter λ . The model provides a simple stochastic baseline: unlike a periodic schedule, it introduces variability while retaining a controllable mean rate.

The independence assumption is not intended to describe every healthcare process. Coordinated access, batches, shifts, and campaigns may produce dependencies and autocorrelation. In the framework, the Poisson process is a design decision for generating reproducible and parameterized workloads; its suitability for real traffic must be evaluated against observed data, which are not currently available in the corpus.

The timeline accepts an intensity function $\lambda(t)$. A constant rate implements the homogeneous case; a variable rate implements a non-homogeneous process through thinning. Candidate events are generated at an upper-bound rate λ_{max} and accepted with probability $\lambda(t)/\lambda_{max}$. A seed controls the pseudo-random generator, making the same timeline reproducible under the same configuration.

Each arrival is associated with a request selected from the existing dataset and receives a unique occurrence identifier. The same temporal model can therefore be applied to different mixes without losing step correlation. Duration is expressed in seconds and rate in arrivals per second; the final number of events remains a random variable and must not be confused with the nominal size of the source dataset.

The experimental protocol and results must report the parameters actually used, sample sizes, and repetitions. This chapter assigns no empirical validity to a particular configuration. [DATA MISSING: observed arrival distribution

in real RVE systems, required to validate the Poisson model against operational traffic.]

3.4.2 Non-Stationary Traffic Profiles

A non-stationary process permits a variable intensity $\lambda(t)$. In an interval $[a, b]$, the expected number of arrivals is

$$\mathbb{E}[N(a, b)] = \int_a^b \lambda(u) du. \quad (3.3)$$

This formulation represents ramps, peak periods, and decreasing phases without abandoning a stochastic model of inter-arrival times.

The framework implements a constant profile, a time-banded synthetic daily clinical profile, and a step profile defined in configuration. The clinical profile is a synthetic pattern: it divides the simulated duration into intervals associated with low, base, and peak levels. It is not derived from Veneto traffic measurements and must not be presented as an observed curve. The step profile is particularly useful for controlled experiments because it makes explicit the times at which offered load changes.

Non-stationarity makes it possible to study adaptation and transients. A system may sustain the same overall mean rate while responding differently to a ramp or sequence of peaks. To interpret this effect, metrics must be aggregated over windows consistent with the scale of change; a mean over the entire execution could erase the phase of interest.

The seed makes the pseudo-random sequence reproducible, but does not make a live execution deterministic because the network and external systems remain variable. The protocol must therefore record the profile, parameters, seed, duration, code version, and configuration, and must include independent repetitions.

3.4.3 Probabilistic Bursts and Overload Scenarios

A burst is a temporary cluster of closely spaced arrivals superimposed on baseline traffic. In the implemented model, each baseline arrival triggers a cluster with probability p_b . Cluster size is sampled between configurable minimum and maximum values; each additional event receives a uniformly distributed offset within a window of width `burst_spread`. Events are marked with their arrival type and burst identifier.

This construction distinguishes a burst from a sustained increase in $\lambda(t)$. The former concentrates requests within a short window; the latter changes the rate over a longer interval. Saturation is not an input parameter, but a behavior to observe when offered load interacts with the concurrency limit and system capacity. A burst may not saturate an overprovisioned system, whereas a sustained rate may produce queuing even without clusters.

Potential effects are visible through several KPIs: generator-side queue delay, active flows and steps, windowed throughput, request latency, flow completion time, error rate, and timeouts. Comparison between burst and non-burst events is descriptive; causal attribution requires control of transaction mix, payload size, and system conditions.

Burst parameters are configurable, and the seed makes injection reproducible. The marker in the normalized dataset supports stratified analysis. Again, the model does not claim that bursts of a particular size correspond to a real clinical process. [DATA MISSING: operational traces or domain statistics required to calibrate the probability, size, and duration of RVE bursts.]

3.4.4 Synthetic Workloads for Distributed Systems

A synthetic workload is a population of operations constructed according to explicit rules and used to exercise a system without relying on personal data or a complete operational trace. Its quality does not derive from volume alone: it must preserve flow structure, type distribution, dependencies among steps, and payload characteristics relevant to the experimental question.

The flow mix assigns a non-negative weight to each flow type. After normalization, the weight represents its selection probability in the workload. Actual counts may differ slightly from theoretical proportions because of sampling and finite population size. The seed and dataset size must therefore be recorded.

The current codebase defines seven flow types: context call (RVE-1.b, RVE-121, RVE-130), patient query (RVE-1.b, RVE-54), patient creation (RVE-1.b, RVE-55), document query (RVE-1.b, ITI-18), document retrieval (RVE-1.b, ITI-43), a mediated patient-registry path (middleware token followed by gateway operations RVE-54, RVE-55, RVE-57, and RVE-100), and a two-step ScrybaSign signature flow. This list describes current software coverage; it does not demonstrate equal functional validation or live execution for every flow.

The timeline does not invent new steps, but samples previously constructed flows. This keeps two experimental dimensions separate: workload composition and temporal process. Arrival profiles can therefore be compared under a constant mix, or different mixes under the same profile, avoiding changes to multiple variables without declaring them.

The synthetic nature of the workload imposes limitations. Patient resources do not represent real individuals; weights are not operational frequencies until calibrated; and mocks do not measure the performance of real services. Its methodological value lies in controllability and reproducibility. Representativeness must be argued separately and declared as a limitation when production data are unavailable.

3.5 Literature on Latency, Throughput, and Scheduling

The three selected studies examine latency from different perspectives. Iorio, Risso, and Casetti conduct end-to-end measurements in task-offloading scenarios; Dimitrov and Skjellum compare microbenchmarks and MPI applications; Akram, Akram, and Rehman formulate a stochastic model for load-sharing decisions Iorio, Risso e Casetti, [«When Latency Matters: Measure-](#)

ments and Lessons Learned»; Dimitrov e Skjellum, «Impact of Latency on Applications' Performance»; Saira Akram, Muhammad Asim Akram e Fattah UR Rehman. «Numerical Evaluation of Network Latency and Throughput in Distributed Systems». In: *The Science Post* 1.4 (dic. 2025). DOI: [10.5281/zenodo.18012513](https://doi.org/10.5281/zenodo.18012513). None of these studies concerns RVE healthcare gateways, and their quantitative results are not directly comparable with the experiments in this thesis.

Their joint reading is nevertheless methodologically useful. The first study shows the multiplicity of contributions to application time; the second constrains the validity of extrapolations from ping-pong tests; and the third highlights that decisions based on delayed information must consider distributions and temporal windows. The following subsections extract these principles while preserving the separate scope of each study and deferring the operational definitions of the adopted KPIs to Chapter 8.

3.5.1 Effects of Latency on Transfer Effectiveness and Throughput

Latency and throughput describe related but non-equivalent dimensions. Latency concerns the duration of an operation or flow; throughput concerns the number of completions per unit of time. An increase in the former may reduce the latter when operations occupy resources or concurrency slots for longer, but the effect depends on the execution model, the possibility of overlapping work, and the available capacity.

In the task-offloading scenario studied by Iorio, Risso, and Casetti, flow completion time includes propagation, transmission, transport, application protocols, and processing. Their experiments show that reducing the distance to the server does not remove the other contributions, and that message size and connection management may alter application time Iorio, Risso e Casetti, «When Latency Matters: Measurements and Lessons Learned». This result supports end-to-end measurement but provides no coefficients that can be transferred to healthcare payloads.

In the MPI-cluster study, the lower point-to-point latency achieved through polling does not improve the considered application benchmarks, which are dominated by medium or large messages and bandwidth. Although asynchronous modes introduce overhead in microbenchmarks, they permit overlap and reduce CPU consumption for communication Dimitrov e Skjellum, «[Impact of Latency on Applications' Performance](#)». The lesson is that a local optimization may not increase application throughput when the actual pattern exploits other mechanisms.

In the load-sharing model by Akram, Akram, and Rehman, latency makes information about the sender or receiver state stale; a transfer may therefore consume resources without producing the intended benefit Akram, Akram e Rehman, «[Numerical Evaluation of Network Latency and Throughput in Distributed Systems](#)». Useful throughput is implicitly distinguished from the simple number of transfers initiated: increasing migration traffic is not equivalent to increasing completed work.

For the RVE gateway, the consequence is to measure both requests and completed flows. A high call rate may include retries, steps destined to fail, or interrupted flows. Flow throughput and flow success rate describe useful work more directly, while request throughput and per-transaction latency help localize behavior. Units and windows must remain explicit, and no causal relationship should be anticipated before the data are available.

3.5.2 Load-Sharing Window, Transfer in Vain, and Transfer Unnecessary

The model by Akram, Akram, and Rehman represents nodes as M/M/1 queues and distinguishes underloaded and overloaded states through thresholds. When an overloaded node identifies an available receiver, there is a random interval during which the transfer may still be useful. This interval is called the Load-Sharing Window (LSW) and is defined with respect to the first relevant state change of either sender or receiver [ibid.](#)

In the model, a *Transfer in Vain* (TIV) occurs when the receiver ceases to be available before the job arrives. A *Transfer Unnecessary* (TU) occurs when the sender clears its own backlog during the transfer. In both cases, the decision was based on a state that is no longer valid when its effect occurs. Latency therefore acts as information age rather than only as an additive cost.

The paper formalizes the window through first-passage times of the queues and evaluates the probability that it remains open beyond the time required for the decision and transfer. The model depends on specific assumptions: Poisson arrivals, exponential services, state thresholds, and job migration. These assumptions do not describe execution of an RVE flow, in which no load sharing between computational queues occurs.

The transferable lesson is narrower: a decision based on remote state must account for the delay between observing and using that information. In the framework, this idea supports recording planned timestamps, actual starts, step durations, and completion time, but does not justify implementing TIV, TU, or LSW as RVE KPIs. Including them in the results without a corresponding model would create a false equivalence.

A further lesson concerns useful throughput. Sending more work may increase overhead and reduce effectiveness when conditions change before completion. In the healthcare case, a cautious analogy is that calls initiated with expired tokens or invalid dependencies do not contribute to successful flows; this remains a methodological analogy, however, rather than an application of the M/M/1 model.

3.5.3 Quantile-Based Decision Rules and the Risks of Simple Means

A mean-based decision uses a central value and disregards the probability of extreme observations. In the LSW model, this simplification is problematic because a decision must remain valid before a random window closes. Akram, Akram, and Rehman propose a rule based on a quantile of the window distribution: a transfer is admitted only when the expected delay is compatible with

a selected confidence level [ibid.](#)

The methodological value of this rule lies in making risk explicit. If q_β represents a threshold derived from the distribution, the parameter β makes the conservativeness of the decision visible. A stricter threshold reduces potentially ineffective transfers but may forgo useful opportunities. The quantile is therefore not inherently superior; it makes the trade-off between exploiting opportunities and risking failure configurable.

The same logic supports the use of percentiles in metrics, but the two uses must not be conflated. In the paper, the quantile enters the scheduling rule; in the RVE framework, P95 and P99 are descriptive statistics computed after execution. The framework does not currently use latency quantiles to admit or reject flows. Applying such a policy would require a design extension and new validation.

Means remain useful for aggregate comparisons and some capacity formulas, but must be accompanied by the median, percentiles, sample size, and temporal variability. In particular, a stable mean may coexist with a high P99 or brief periods of errors. The selection of statistics must be driven by the question: typical behavior, distribution tail, reliability, or adaptation to variable load.

The numerical paper reports improvements within its simulated scenario, but these values depend on the M/M/1 model and adopted parameters and are not used as predictions for SI.Ter or the framework. The evidence relevant to this thesis is limited to the principle that, when risk lies in the distribution tail, a simple mean is not a sufficient basis for decision-making.

3.5.4 Local Latency Optimization and End-to-End Performance

A local optimization reduces the time of one component or step. End-to-end performance instead depends on the composition of all steps, their dependencies, and the load. If the optimized component contributes little to the critical path, the final benefit may be limited; if the optimization consumes a shared resource, it may even worsen other dimensions.

The three studies provide complementary examples. Moving the server closer reduces propagation but does not remove transmission, protocol, and processing time Iorio, Risso e Casetti, «[When Latency Matters: Measurements and Lessons Learned](#)». Reducing MPI latency through polling does not necessarily improve bandwidth-dominated applications and may increase CPU use Dimitrov e Skjellum, «[Impact of Latency on Applications' Performance](#)». Rapidly executing a migration decision does not make it useful if load information is already stale Akram, Akram e Rehman, «[Numerical Evaluation of Network Latency and Throughput in Distributed Systems](#)».

In the interoperability case, optimizing a single transaction may not reduce completion time proportionally when authentication, document retrieval, or queuing dominates the flow. Similarly, reducing gateway latency does not demonstrate improvement of the overall service when the backend or access network remains dominant. Identifying the contribution requires measurements at the correct boundaries and controlled scenarios.

This principle motivates the framework's dual granularity: step and flow. Per-step metrics expose local differences, while flow completion time tests whether those differences affect the overall outcome. Middleware data, when available, add another level but do not automatically authorize a causal decomposition.

3.6 Implications for Framework Design

The preceding sections converge on four methodological requirements. Healthcare semantics require complete sequences and dependencies among steps to be represented. The distributed nature of the system requires client and middleware boundaries to remain distinct. Latency variability requires structured logs, percentiles, and time series. Finally, capacity evaluation requires controllable, non-stationary, and reproducible load profiles.

This section states how these principles guide the current framework without replacing the design and implementation chapters. The described choi-

ces have been verified in the codebase, while their performance effects remain subject to the experimental protocol. No feature is presented as a result, and no available metric is interpreted as evidence until it has been applied to real experimental datasets.

3.6.1 Measuring Complete Flows Rather Than Individual Calls

An individual HTTP call is a technical unit; a flow is a process unit. In the cases studied, the outcome depends on ordering and output propagation: the assertion precedes the access-token request, the token precedes the protected call, and the document query precedes document retrieval. Measuring calls alone would omit the cost of composition and the consequences of intermediate failures.

The framework represents each flow type as an ordered list of steps. Each occurrence receives a distinct identifier, while its steps share the flow ID and have their own request ID and index. In live mode, placeholders in dependent steps are resolved from outputs extracted from preceding responses; in mock mode, synthetic outputs are prepared within the chain. This structure preserves data dependencies without sharing context among occurrences.

The execution engine runs the steps of a flow sequentially and stops the chain at the first blocking error. Independent flows are scheduled concurrently within a semaphore that limits the number of active occurrences. The invariant is therefore explicit: internal sequencing and external concurrency. This behavior is consistent with the requirement not to use tokens or identifiers before they become available.

Measurements are required at least at two granularities. Client time, outcome, size, and transaction type are recorded for each step. Total duration, success, failure step, and number of steps actually executed are recorded for the flow. The normalized dataset also preserves steps skipped after a failure, preventing them from being interpreted as requests absent for incidental reasons.

This choice does not imply that flow completion time is always the sum of step latencies. Placeholder resolution, scheduling, and local code contribute to the overall interval. Comparing the sum with total duration may be diagnostically useful, but must not be imposed as an identity. Likewise, failed flows must be separated from successful completions when distributions are constructed.

3.6.2 Separating Measurement Levels Without Forcing Unsupported Correlations

The framework adopts a two-level grey-box approach. The framework layer observes what the generator can measure directly: planned and actual timestamps, requests, responses, client latency, concurrency, and flow completion. When available, the middleware layer may add gateway latency, upstream latency, retries, circuit-breaker state, and infrastructure indicators.

This separation follows from a limitation discussed in the literature: an end-to-end measurement represents application experience but does not automatically localize its cause Iorio, Risso e Casetti, «[When Latency Matters: Measurements and Lessons Learned](#)». Internal data increase diagnostic capability while introducing correlation requirements. A middleware record must be associated with the correct occurrence and corresponding step; equality of a reused identifier alone may be insufficient.

The collector normalizes framework records even without middleware data. If a second log is present, the join enriches matching requests; otherwise, internal fields remain null. This makes coverage explicit and prevents simulated or inferred values from replacing missing observations. Join completeness is itself an observability metric.

The mock middleware log is a pipeline-testing instrument, not a measurement of real middleware. Its synthetic latency decomposition is used to verify parsers, joins, and reports. The data it produces cannot support conclusions about the gateway, backend, or infrastructure. In the results, they must be marked as mock data and kept separate from live executions.

Statistical correlation among CPU, connections, and latency does not demonstrate causality either. It may suggest a hypothesis to test through controlled load variation, repetitions, and control of external variables. The generalizable guidelines are therefore to keep boundaries separate, measure correlation coverage, and state the origin of every field. Availability of internal data remains specific to the environment and logging policies.

3.6.3 Collecting Structured and Normalized Logs

A reproducible experiment requires machine-readable events rather than diagnostic messages alone. The framework emits three JSON Lines (JSONL) event types, with one JSON object per line. A `request_sent` event records dispatch; a `response_received` event records the corresponding outcome; and a `flow_completed` event records flow completion. This format supports incremental processing and preserves event granularity without enclosing all records in one monolithic document.

The events share experiment and correlation metadata: run, scenario, execution, flow occurrence, flow, request, and step. A sent-request event includes planned and actual times, queue delay, concurrency, arrival type, and payload size. A response event includes status, success, error, size, and client latency. A completion event includes duration, outcome, and failure step. These fields are described here to motivate traceability; the complete schema belongs to the implementation chapter.

The collector pairs send and response events by occurrence, constructs request-level records, reconstructs flows, and aggregates a bucketed time series. If a middleware log is available, it adds internal fields without overwriting the client measurement. The normalized result separates `requests`, `flows`, and `timeseries`, thereby preserving the aggregation levels required by the methodology.

Normalization addresses two problems. First, source logs represent events, whereas metrics require complete records. Second, framework and middleware

logs may use different schemas. The join constructs a common view but must preserve null values and data provenance so that absence is not interpreted as zero. Malformed or uncorrelatable records must be counted and reported rather than completed by assumption.

Confidentiality is a cross-cutting requirement. Experimental logs must exclude tokens, assertions, credentials, and personal data. Synthetic identifiers and technical metadata are sufficient for event correlation. This choice separates performance observability from clinical audit: the framework does not replace production audit infrastructure and must not reproduce its sensitive contents.

3.6.4 Evaluating Behavior Under Realistic Loads, Bursts, and Flow Mixes

A workload suitable for evaluation must be realistic with respect to the structure of the problem and controllable with respect to the experiment. In the framework, structural realism derives from flow types and payloads conforming to the specifications studied; temporal realism is instead a configurable hypothesis expressed through rate, profile, and bursts. Without operational traces, the term *realistic* denotes plausibility and controlled diversity rather than demonstrated equivalence with production.

The flow mix determines the selection probability of each path. Keeping the mix constant isolates the effect of the load profile, while changing a single weight supports composition-based comparisons. Because flows have different numbers of steps and payloads, the same flow rate may produce a different request rate. Both quantities must be reported.

The constant profile provides a baseline; the step profile permits ramps and sustained levels; the synthetic clinical profile introduces time bands; and bursts superimpose short clusters on baseline traffic. For each scenario, the duration, $\lambda(t)$ function or steps, seed, burst probability, minimum and maximum cluster size, spread, and concurrency limit must be recorded. Configurations present

in the repository do not automatically constitute executed experiments.

Temporary peaks, sustained load, and saturation must be distinguished operationally. A peak is a brief increase in offered load; sustained load maintains a level over a declared window; saturation may be inferred only from combined evidence, such as persistent growth in queue delay, concurrency at the configured limit, a throughput plateau, and a possible increase in errors or timeouts. No single indicator demonstrates saturation by itself.

The metric engine compares burst and non-burst distributions, produces throughput and error rates over time, and observes concurrency and scheduling drift. These capabilities prepare the protocol but do not demonstrate gateway behavior. Chapter 9 must use only logs produced by declared executions, report populations and windows, and refrain from quantitative conclusions when data are insufficient.

[DATA MISSING: calibration of flow mixes and temporal profiles using operational RVE statistics; until such data become available, the scenarios remain controlled synthetic workloads.]

Technical and Functional Analysis

4.1 Casi d'uso

Per lo studio dei casi di utilizzo del prodotto sono stati creati dei diagrammi. I diagrammi dei casi d'uso (in inglese *Use Case Diagram*) sono diagrammi di tipo *Unified Modeling Language (UML)*_G dedicati alla descrizione delle funzioni o servizi offerti da un sistema, così come sono percepiti e utilizzati dagli attori che interagiscono col sistema stesso. Essendo il progetto finalizzato alla creazione di un tool *Nome del termine*_G per l'automazione di un processo, le interazioni da parte dell'utilizzatore devono essere ovviamente ridotte allo stretto necessario. Per questo motivo i diagrammi d'uso risultano semplici e in numero ridotto.

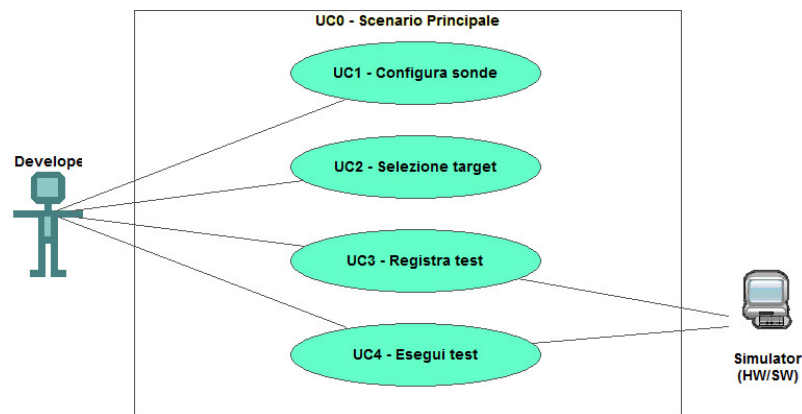


Figura 4.1: Use Case 0: Scenario principale

UC0: Scenario principale

Attori Principali: Sviluppatore applicativi.

Precondizioni: Lo sviluppatore è entrato nel plugin di simulazione all'interno dell'IDE.

Descrizione: La finestra di simulazione mette a disposizione i comandi per

configurare, registrare o eseguire un test.

Postcondizioni: Il sistema è pronto per permettere una nuova interazione.

UC1: Gestione Utente

Attori Principali: Amministratore, Utente Registrato.

Precondizioni: L'utente deve essere autenticato nel sistema.

Descrizione: L'utente può gestire le informazioni del proprio profilo.

Postcondizioni: Le modifiche vengono salvate nel sistema.

Scenario Alternativo: Se l'utente non è autenticato, visualizza un messaggio di errore.

UC2: Creazione Prodotto

Attori Principali: Amministratore.

Precondizioni: L'amministratore ha effettuato l'accesso al sistema.

Descrizione: L'amministratore può aggiungere un nuovo prodotto al catalogo.

Postcondizioni: Il nuovo prodotto viene aggiunto con successo.

Scenario Alternativo: Se i campi obbligatori non sono compilati, visualizza un messaggio di errore.

4.2 Tracciamento dei requisiti

Da un'attenta analisi dei requisiti e degli use case effettuata sul progetto è stata stilata la tabella che traccia i requisiti in rapporto agli use case.

Sono stati individuati diversi tipi di requisiti e si è quindi fatto utilizzo di un codice identificativo per distinguerli.

Il codice dei requisiti, dove ogni requisito è identificato con il carattere **R**, è così strutturato:

F: Funzionale.

Q: Qualitativo.

V: Di vincolo.

N: Obbligatorio (necessario).

D: Desiderabile.

Z: Opzionale.

Nelle tabelle ??, ?? e ?? sono riassunti i requisiti e il loro tracciamento con gli use case delineati in fase di analisi.

4.3 Tabelle dei requisiti

	Requisito	Descrizione	Use Case
RFN-1	L'interfaccia permette di configurare il tipo di sonde del test		UC1

Tabella 4.1: Tabella del tracciamento dei requisiti funzionali.

	Requisito	Descrizione	Use Case
RQD-1n	Le prestazioni del simulatore hardware deve garantire la giusta esecuzione dei test e non la generazione di falsi negativi	-	
RQD-1n	Le prestazioni del simulatore hardware deve garantire la giusta esecuzione dei test e non la generazione di falsi negativi	-	
RQD-1n	Le prestazioni del simulatore hardware deve garantire la giusta esecuzione dei test e non la generazione di falsi negativi	-	
	Continua nella prossima pagina...		

Tabella 4.2 – Continuo della tabella

	Requisito	Descrizione	Use Case
RQD-1n	Le prestazioni del simulatore hardware deve garantire la giusta esecuzione dei test e non la generazione di falsi negativi	-	
RQD-1n	Le prestazioni del simulatore hardware deve garantire la giusta esecuzione dei test e non la generazione di falsi negativi	-	
RQD-1n	Le prestazioni del simulatore hardware deve garantire la giusta esecuzione dei test e non la generazione di falsi negativi	-	

Tabella 4.2: Tabella del tracciamento dei requisiti qualitativi.

	Requisito	Descrizione	Use Case
RVO-1	La libreria per l'esecuzione dei test automatici deve essere riutilizzabile	-	

Tabella 4.3: Tabella del tracciamento dei requisiti di vincolo.

Capitolo 5

Architecture and Transactions

Breve introduzione al capitolo

5.1 Tecnologie e strumenti

Di seguito viene data una panoramica delle tecnologie e strumenti utilizzati.

Tecnologia 1

Descrizione Tecnologia 1.

Tecnologia 2

Descrizione Tecnologia 2

5.2 Ciclo di vita del software

5.3 Progettazione

5.3.1 Namespace 1

Descrizione namespace 1.

5.4 Design Pattern utilizzati

5.5 Codifica

Blocco di codice in C

```
#include <stdio.h>
int main() {
    print("Hello, world!");
    return 0;
}
```

Codice 5.1: Example of code

Capitolo 6

Framework Design

Capitolo 7

Framework Implementation

Bibliografia

Testi

James P. Womack, Daniel T. Jones. *Lean Thinking, Second Editon*. Simon & Schuster, Inc., 2010.

Articoli

Akram, Saira, Muhammad Asim Akram e Fattah UR Rehman. «Numerical Evaluation of Network Latency and Throughput in Distributed Systems». In: *The Science Post* 1.4 (dic. 2025). DOI: [10.5281/zenodo.18012513](https://doi.org/10.5281/zenodo.18012513).

Einstein, Albert, Boris Podolsky e Nathan Rosen. «Can Quantum-Mechanical Description of Physical Reality be Considered Complete?» In: *Physical Review* 47.10 (1935), pp. 777–780. DOI: [10.1103/PhysRev.47.777](https://doi.org/10.1103/PhysRev.47.777).

Iorio, Marco, Fulvio Risso e Claudio Casetti. «When Latency Matters: Measurements and Lessons Learned». In: *ACM SIGCOMM Computer Communication Review* 51.4 (ott. 2021).

Sitografia

- HL7 International. *FHIR Release 4 (Version 4.0.1): Overview*. URL: <https://hl7.org/fhir/R4/overview.html> (visitato il 22/07/2026).
- *FHIR Release 4 (Version 4.0.1): Resource Bundle*. URL: <https://hl7.org/fhir/R4/bundle.html> (visitato il 22/07/2026).
- IHE International. *Cross-Enterprise Document Sharing (XDS.b)*. URL: <https://profiles.ihe.net/ITI/TF/Volume1/ch-10.html> (visitato il 22/07/2026).
- *Internet User Authorization (IUA)*. URL: <https://profiles.ihe.net/ITI/IUA/> (visitato il 22/07/2026).
- Manifesto Agile*. URL: <http://agilemanifesto.org/iso/it/>.